

QUINTAURIS



Parte I: De la nube a la
FPGA: flujo de despliegue
de una IP en la FPGA

Encarnación Castillo Morales
Luis Parrilla Roure

Departamento de Electrónica y
Tecnología de Computadores. Universidad
de Granada



Agenda



SEMANA 1

LUN 7
17:00–19:00

1. Fundamentos de RISC-V: de la arquitectura al ecosistema
Quintauris · Ángel Berrio

MAR 8
17:00–19:00

2. Presentación de Quintauris: visión, modelo de negocio y productos
Quintauris · Ángel Berrio

MIÉ 9
17:00–19:00

3. Explorando la plataforma Quintauris, Tools and Infra — incluyendo Testing, Benchmarking y QNTAF (I)
Quintauris · Enrique Pallares

JUE 10
17:00–19:00

4. Explorando la plataforma Quintauris, Tools and Infra — incluyendo Testing, Benchmarking y QNTAF (II)
Quintauris · Enrique Pallares

SEMANA 2

2 Semanas - 8 sesiones

LUN 14
16:30–19:30

5. Verificación y validación: del requisito a la evidencia, en software y en silicio
Quintauris · Ignacio Ruiz

MAR 15
16:30–19:30

6. Parte I: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA
UGR & Quintauris · José Luis Martínez / UGR

MIÉ 16
16:30–19:30

7. Parte II: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA
UGR & Quintauris · José Luis Martínez / UGR

JUE 17
16:00–20:00

8. Hackathon final: integración de conceptos y solución de un reto práctico
Quintauris · José Luis Martínez, con mentores de UGR

Objective



From an open-source RISC-V IP repository to a running FPGA-based hardware design.

1. Build a RISC-V-based digital system in VHDL

- Integrate and configure the NEORV32 processor on the Arty A7-100T FPGA board.

2. Use Vivado for hardware implementation

- Synthesize the design, implement it on the FPGA, and generate the bitstream.

3. Program and test the FPGA

- Load the bitstream and verify the system through GPIO and UART.

4. Establish a UART connection with the NEORV32 core

- Configure Tera Term and communicate with the NEORV32 bootloader.

5. Load and execute RISC-V applications

- Compile a C application using the appropriate RISC-V toolchain, upload the executable through UART, and execute it using the bootloader.



NEORV32 Customizable RISC-V processor

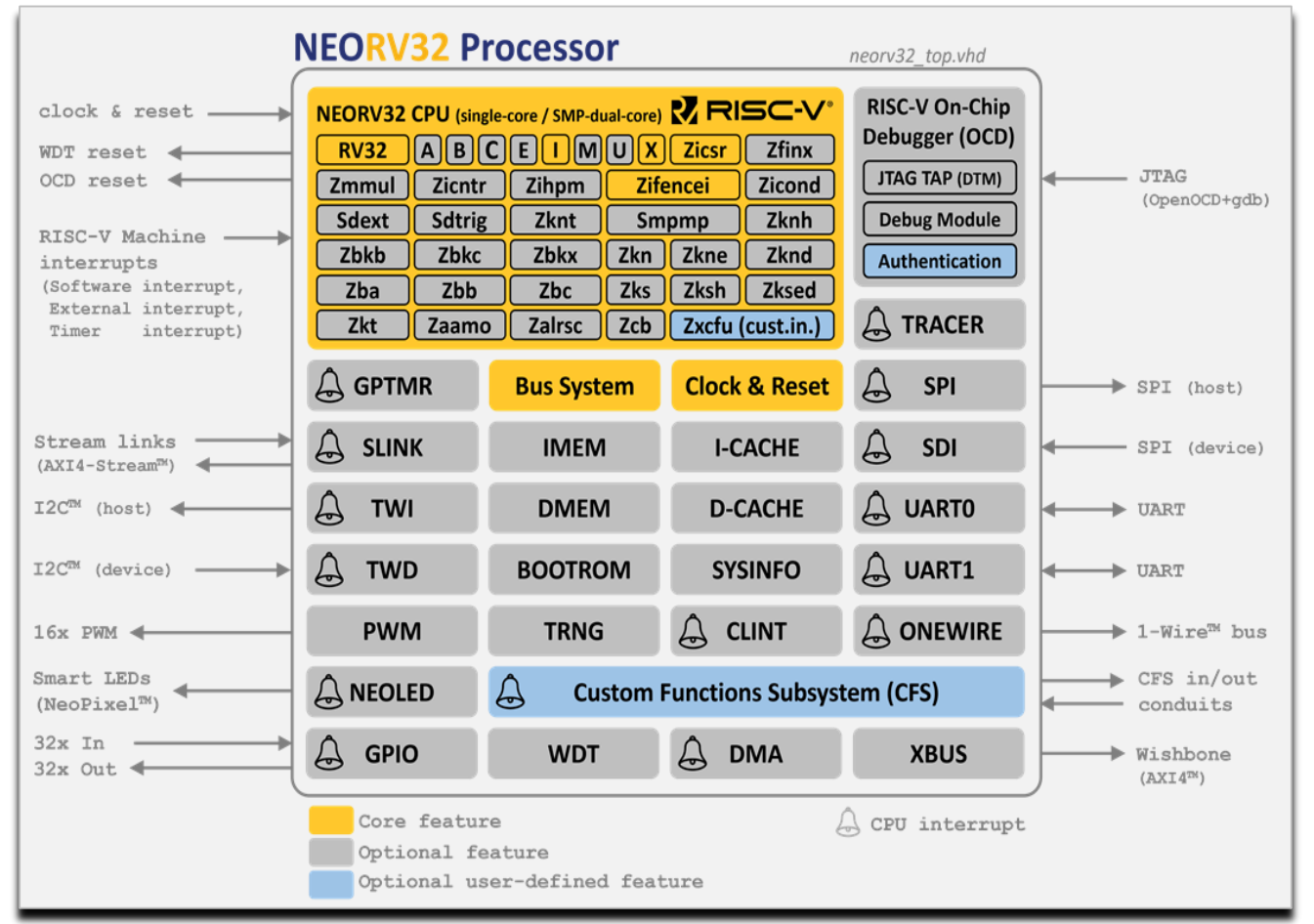


General Structure



Key features

- All-in-one package: CPU + SoC + Software Framework + Tooling
- Completely described in behavioral, platform-independent VHDL - no primitives, macros, attributes, etc.
- Extensive CPU & SoC configuration options for adapting to application requirements
- Aims to be as small as possible while being as RISC-V-compliant as possible
- FPGA friendly (e.g. all internal memories can be mapped to block RAM)
- Optimized for high clock frequencies to ease integration and timing closure
- From zero to printf("hello world"); - completely open-source and documented
- Easy to use – intended to work out of the box



NEORV32 repository



Download and decompress the repository (Download .zip) or clone it

URL: <https://github.com/stnolting/neorv32>

 stnolting	Rework bootloader; add optional SD card boot option (#1361) ✓	e2131e6 · 5 hours ago	 9,147 Commits
 .github	[ci] revert to GHDL nightly	5 months ago	
 docs	[docs] update section "bootloader"	16 hours ago	
 rtl	[rtl] update default bootloader image	16 hours ago	
 sim	[rtl] update FIFO instances	2 weeks ago	
 sw	[bootloader] update main	17 hours ago	
 .gitignore	[.gitignore] ignore GHDL waveforms	3 weeks ago	
 CHANGELOG.md	[changelog] add v1.12.1.1	16 hours ago	
 CODE_OF_CONDUCT.md	[docs] minor cleanups	last year	
 LICENSE	Update LICENSE	9 months ago	
 README.md	[docs] update bootloader references	2 days ago	

NEORV32 repository



Structure of the repository

```
neorv32          - Project home folder
|
├─ docs          - Documentation
|   ├── datasheet - AsciiDoc sources for the NEORV32 data sheet
|   ├── figures   - Figures and logos
|   ├── references - Data sheets and RISC-V specs
|   └─ userguide  - AsciiDoc sources for the NEORV32 user guide
|
├─ rtl           - HDL sources
|   ├── core      - Core sources of the CPU & SoC
|   ├── processor_templates - Pre-configured SoC wrappers
|   ├── system_integration - System wrappers and bridges for advanced connectivity
|   └─ test_setups - Minimal test setup "SoCs" used in the User Guide
|
├─ sim           - Simulation files
|
└─ sw            - Software framework
    ├── bootloader - Sources of the processor-internal bootloader
    ├── common     - Linker script, crt0.S start-up code and central makefile
    ├── example    - Example programs for the core and the SoC modules
    │   ├── eclipse - Pre-configured Eclipse IDE project
    │   └─ ...      - Several example programs
    ├── image_gen  - Helper program to generate executables & memory images
    ├── lib        - Processor core library
    │   ├── include - NEORV32 core library header files (*.h)
    │   └─ source   - NEORV32 core library source files (*.c)
    ├── ocd_firmware - Firmware for the on-chip debugger "park loop"
    ├── openocd    - OpenOCD configuration files
    └─ svd         - Processor system view description file (CMSIS-SVD)
```

NEORV32 repository



Folders for the Vivado Project

```
neorv32          - Project home folder
|
├─ docs          - Documentation
|   ├── datasheet - AsciiDoc sources for the NEORV32 data sheet
|   ├── figures   - Figures and logos
|   ├── references - Data sheets and RISC-V specs
|   └─ userguide  - AsciiDoc sources for the NEORV32 user guide
|
├─ rtl           - HDL sources
|   └─ core       - Core sources of the CPU & SoC
|
|   ├── processor_templates - Pre-configured SoC wrappers
|   ├── system_integration  - System wrappers and bridges for advanced connectivity
|   └─ test_setups          - Minimal test setup "SoCs" used in the User Guide
|
├─ sim           - Simulation files
|
└─ sw            - Software framework
    ├── bootloader - Sources of the processor-internal bootloader
    ├── common      - Linker script, crt0.S start-up code and central makefile
    ├── example     - Example programs for the core and the SoC modules
    │   ├── eclipse - Pre-configured Eclipse IDE project
    │   └─ ...      - Several example programs
    ├── image_gen   - Helper program to generate executables & memory images
    ├── lib         - Processor core library
    │   ├── include - NEORV32 core library header files (*.h)
    │   └─ source   - NEORV32 core library source files (*.c)
    ├── ocd_firmware - Firmware for the on-chip debugger "park loop"
    ├── openocd      - OpenOCD configuration files
    └─ svd           - Processor system view description file (CMSIS-SVD)
```


NEORV32 repository



Some files in the `core` folder for the Vivado Project

File / Group

`neorv32_package.vhd`

`neorv32_cpu.vhd`

`neorv32_top.vhd`

`neorv32_imem*.vhd`

`neorv32_dmem*.vhd`

`neorv32_gpio.vhd`

`neorv32_uart.vhd`

`neorv32_sys.vhd`

`neorv32_bootrom*.vhd`

Other `neorv32_*.vhd` files

Function

Package with types, constants, and common definitions.

RISC-V CPU.

Top level of the NEORV32 processor/SoC.

Instruction memory.

Data memory.

GPIO interface.

UART interface.

System infrastructure and reset logic.

Bootloader / start-up ROM.

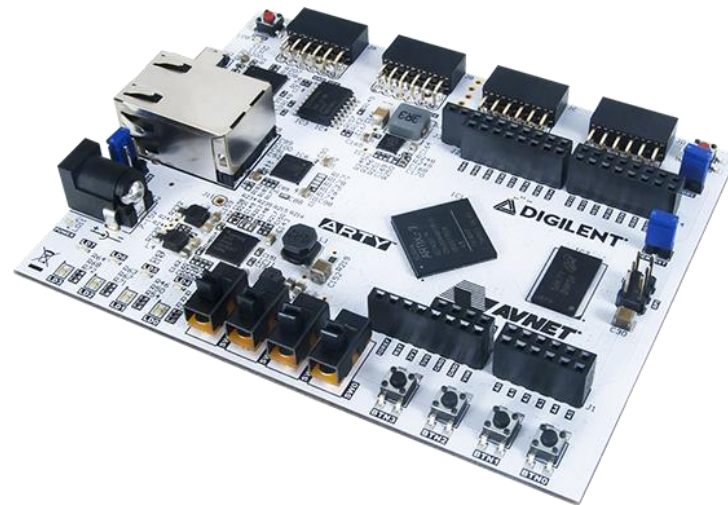
Other peripherals and SoC blocks.



Digilent Arty A7-100T Development board

FPGA AMD/Xilinx Artix-7

<https://digilent.com/reference/programmable-logic/arty-a7/start>

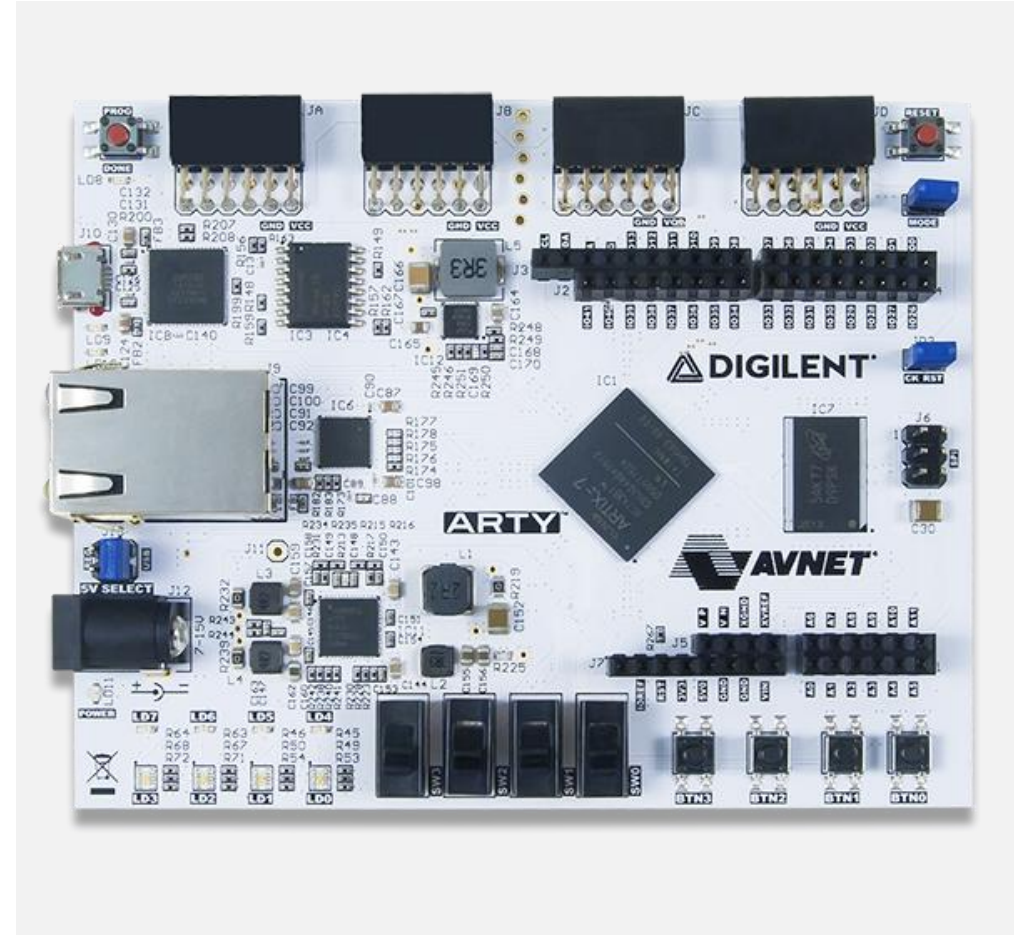


Artix-7 FPGA Development Board



Key features (I)

- FPGA Part XC7A100TCSG324-1
- Logic Slices 15,850
- Block RAM 4,860 Kbits
- DSP Slices 240
- DDR3 256 MB @ 333 MHz (667 MT/s)
- Internal clock 450 MHz+
- Quad-SPI Flash 16 MB
- Ethernet 10/100 Mbps

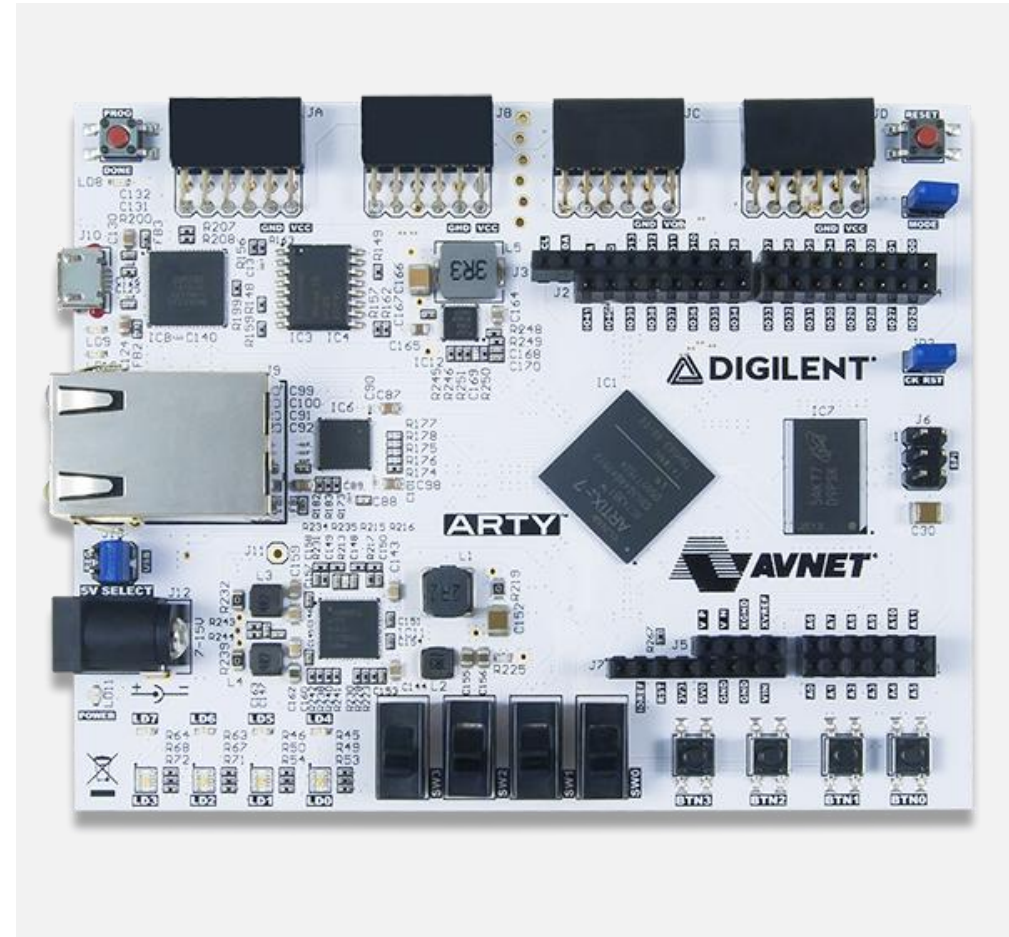


Artix-7 FPGA Development Board



Key features (II)

- Connectivity and Onboard I/O
- Pmod Connectors 4
- Shield Connector
- Arduino / chipKIT compatible
- Switches 4
- Buttons 4
- Reset LED 1
- User LED 4
- User RGB LED 4
- Electrical Power USB, 7-15v (2.5mm coaxial) supply
- Logic Level 3.3v





AMD Vivado

v.2026.1: <https://www.xilinx.com/support/download.html>

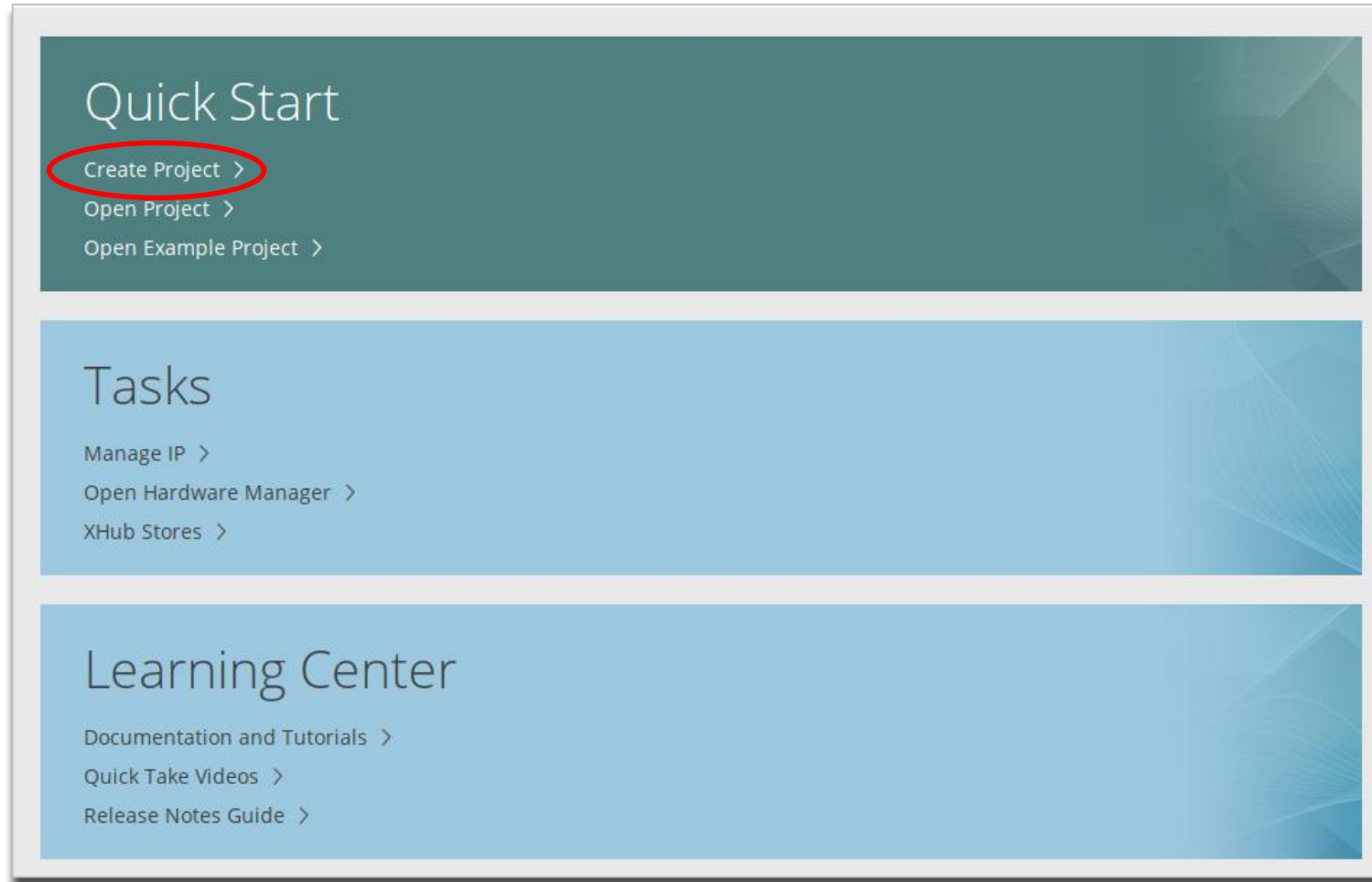
v.2025.1: <https://www.amd.com/en/support/downloads/adaptive-socs-and-fpgas/development-tools/2025-1.html>



Vivado project generation



Starting window



Vivado project generation



Creating project (I)

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:

☒ **Create project subdirectory**

Project will be created at: C:/Users/Usuario/Documents/investigacion/vivado_projects/neorv32_quintauris

New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☒ **Do not specify sources at this time**

☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and

Vivado project generation



Creating project (II)

New Project

Default Part

Choose a default AMD part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: Name: Board Rev:

Search:

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev
Arty			digilentinc.com	1.1			
Genesys2			digilentinc.com	1.1			
Nexys4			digilentinc.com	1.1			

Refresh

Catalog was last updated on 09/04/2025 2:01:47 PM

?

Back

Next

Finish

Cancel

Vivado project generation



Creating project (III)

New Project

Default Part
Choose a default AMD part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: Name: Board Rev:

Search:

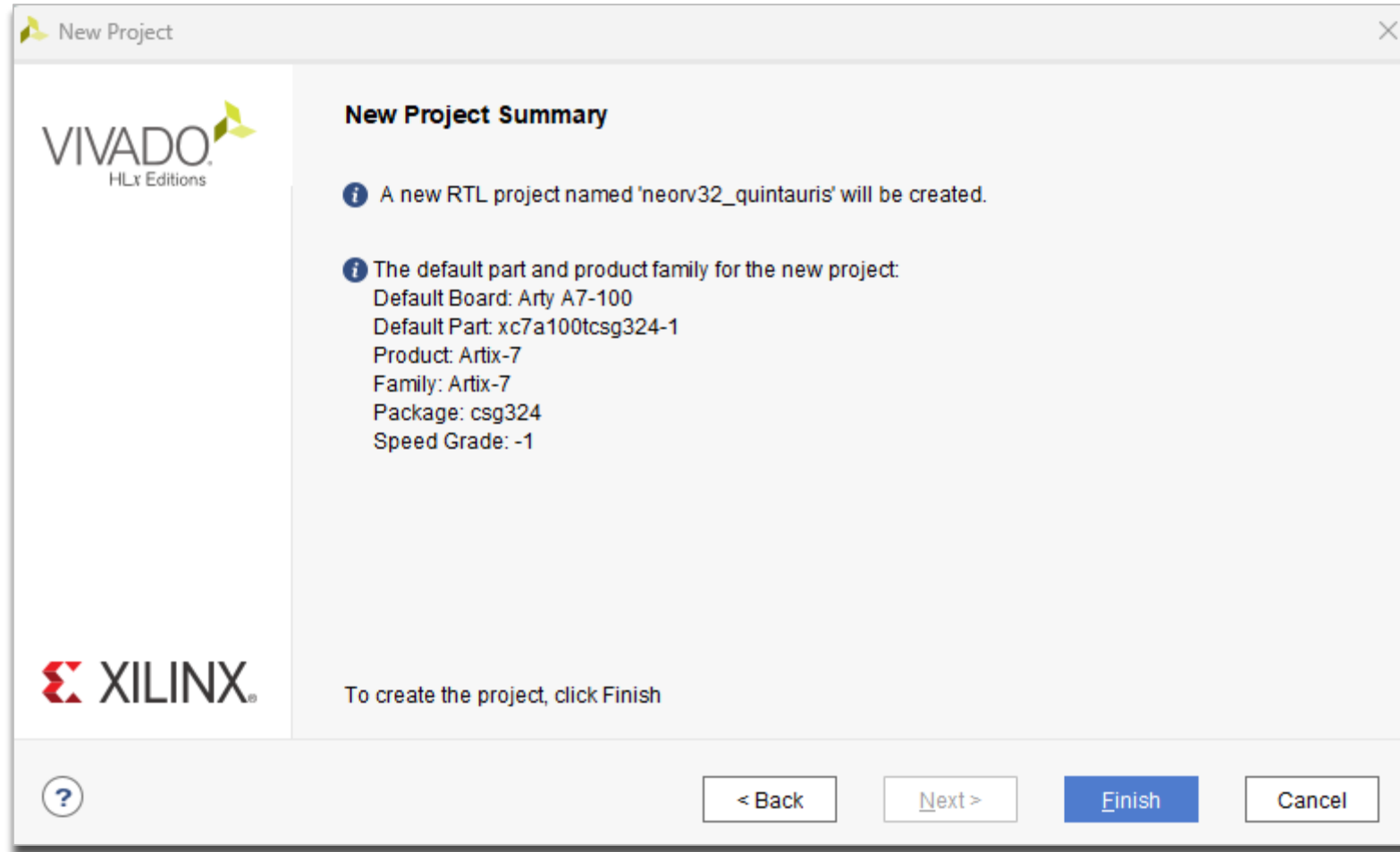
Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev	Available IOBs	LUT Elements	FlipFlops	Block RAMs
Zybo			digilentinc.com	1.0							
Arty A7-100			digilentinc.com	1.1							
Arty A7-35			digilentinc.com	1.1							

Catalog was last updated on 09/04/2025 2:01:47 PM

Vivado project generation



Creating project (IV)



Vivado environment



neorv32_quintauris - [C:/Users/Usuario/Documents/investigacion/vivado_projects/neorv32_quintauris/neorv32_quintauris.xpr] - Vivado 2020.2

File Edit Flow Tools Reports Window Layout View Help Quick Access

Ready

Default Layout

Flow Navigator

PROJECT MANAGER - neorv32_quintauris

PROJECT MANAGER

- Settings
 - Add Sources
 - Language Templates
 - IP Catalog
- IP INTEGRATOR
 - Create Block Design
 - Open Block Design
 - Generate Block Design
- SIMULATION
 - Run Simulation
- RTL ANALYSIS
 - Open Elaborated Design
- SYNTHESIS
 - Run Synthesis
 - Open Synthesized Design
- IMPLEMENTATION
 - Run Implementation
 - Open Implemented Design
- PROGRAM AND DEBUG
 - Generate Bitstream
 - Open Hardware Manager

Sources

- Design Sources
- Constraints
- Simulation Sources
 - sim_1
- Utility Sources

Hierarchy Libraries Compile Order

Properties

Project Summary

Overview | Dashboard

Settings Edit

Project name: neorv32_quintauris

Project location: C:/Users/Usuario/Documents/investigacion/vivado_projects/neorv32_quintauris

Product family: Artix-7

Project part: [Arty A7-100 \(xc7a100tcsg324-1\)](#)

Top module name: [Not defined](#)

Target language: [Verilog](#)

Simulator language: [Mixed](#)

Board Part

Display name: Arty A7-100

Board part name: [digilentinc.com:arty-a7-100:part0:1.0](#)

Board revision: E.0

Connectors: No connections

Repository path: C:/Xilinx/Vivado/2020.2/data/boards/board_files

URL: <https://reference.digilentinc.com/reference/programmable-logic/arty-a7/start>

Board overview: Arty A7-100

Synthesis

Status: Not started

Messages: No errors or warnings

Part: xc7a100tcsg324-1

Strategy: [Vivado Synthesis Defaults](#)

Report Strategy: [Vivado Synthesis Default Reports](#)

Incremental synthesis: [None](#)

Implementation

Status: Not started

Messages: No errors or warnings

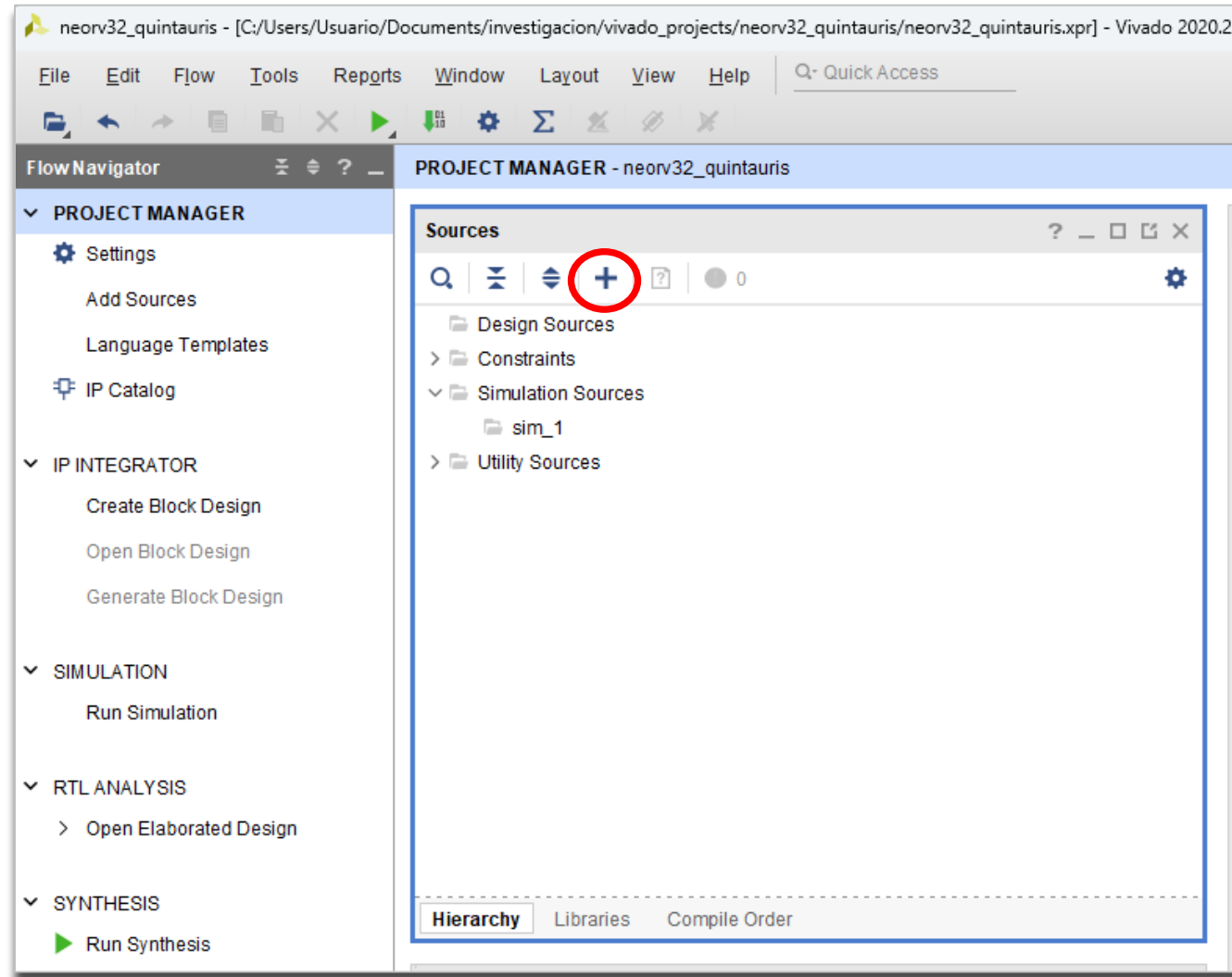
Part: xc7a100tcsg324-1

Strategy: [Vivado Implementation Defaults](#)

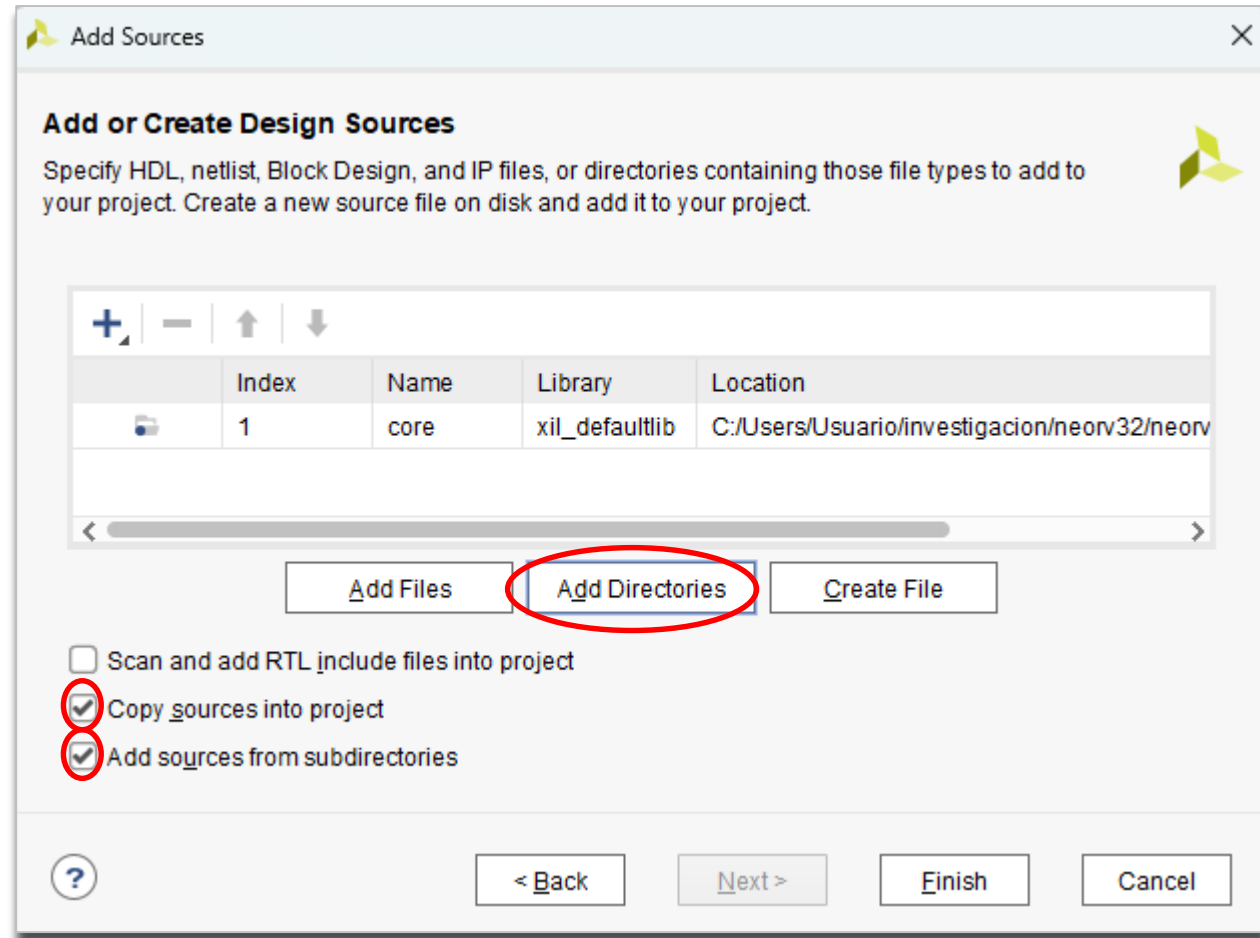
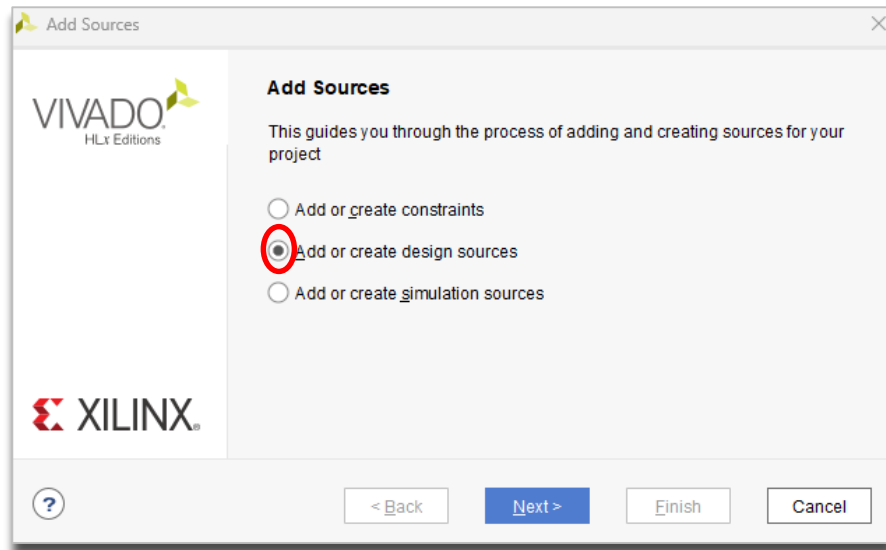
Report Strategy: [Vivado Implementation Default Reports](#)

Incremental implementation: [None](#)

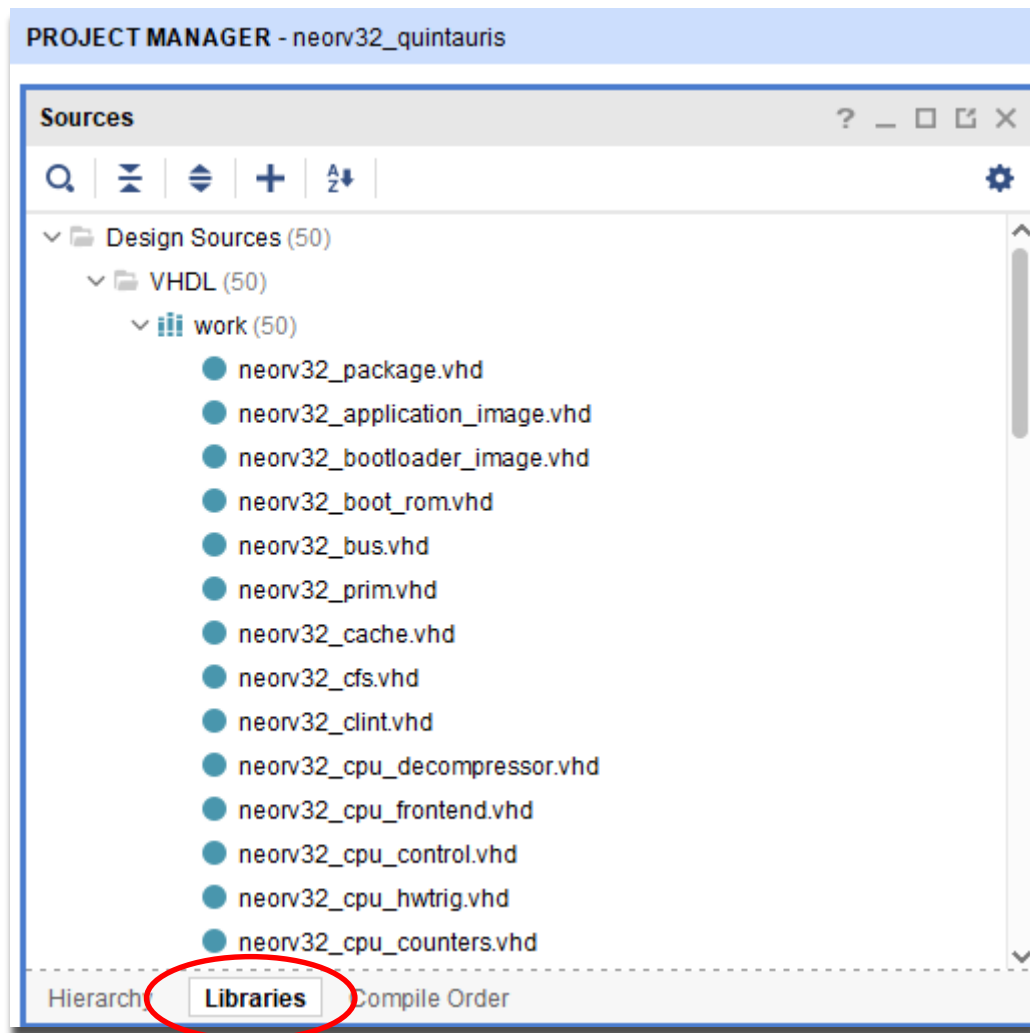
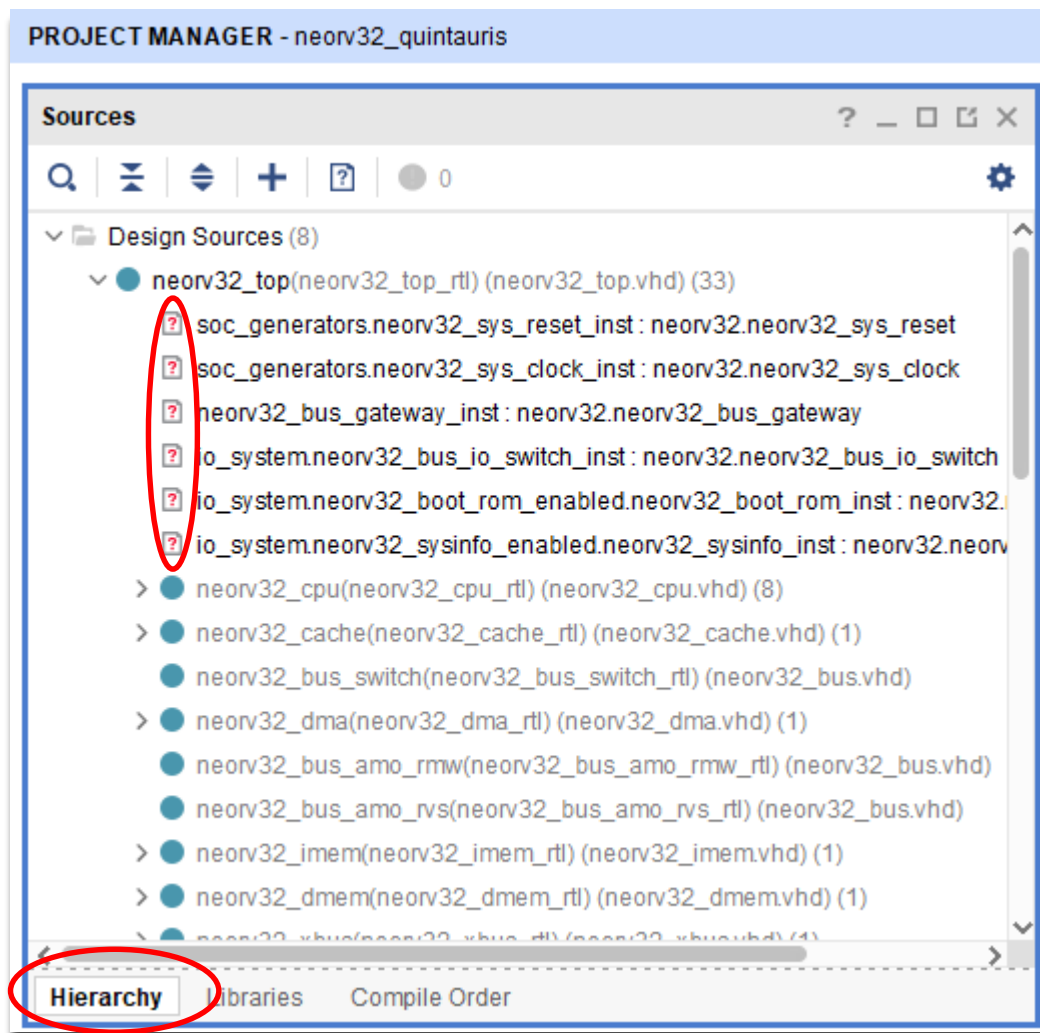
Adding NEORV32 sources to project (I)



Adding NEORV32 sources to project (II)



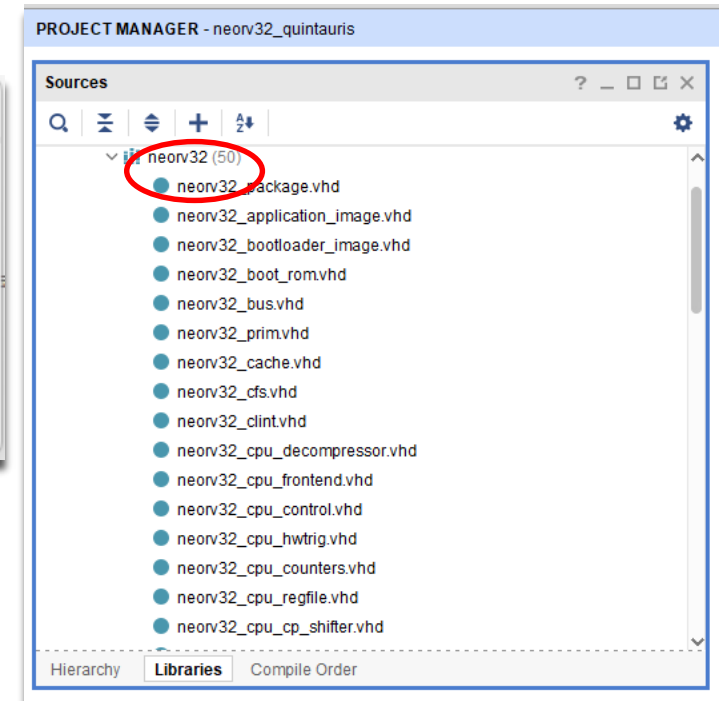
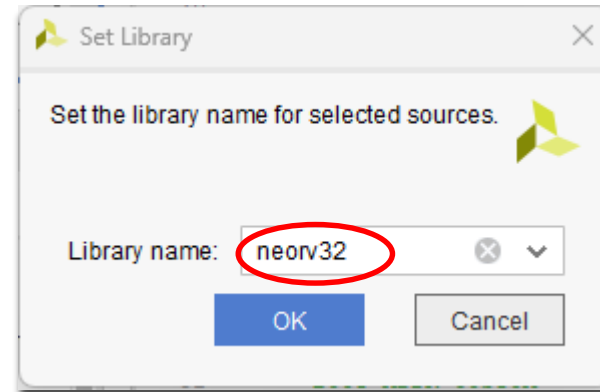
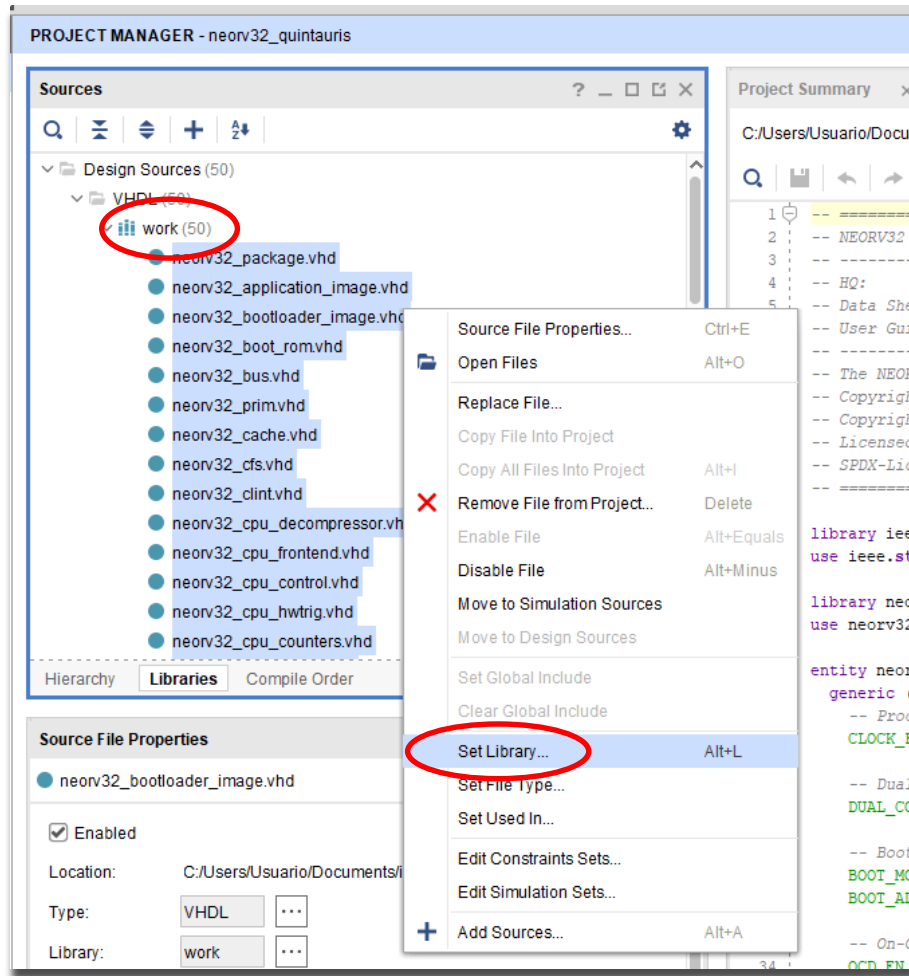
Set NEORV32 library to files (I)



Set NEORV32 library to files (II)

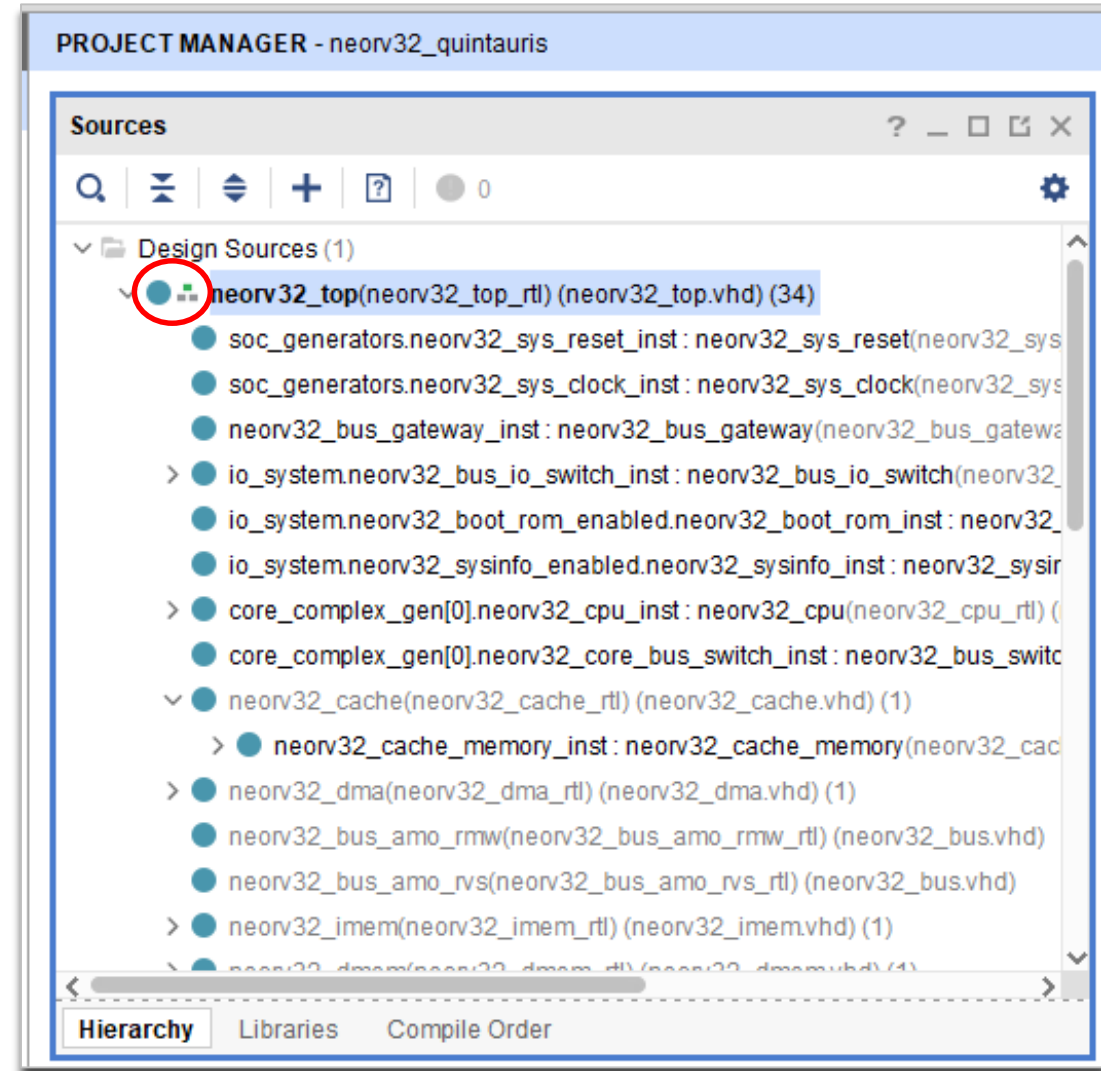
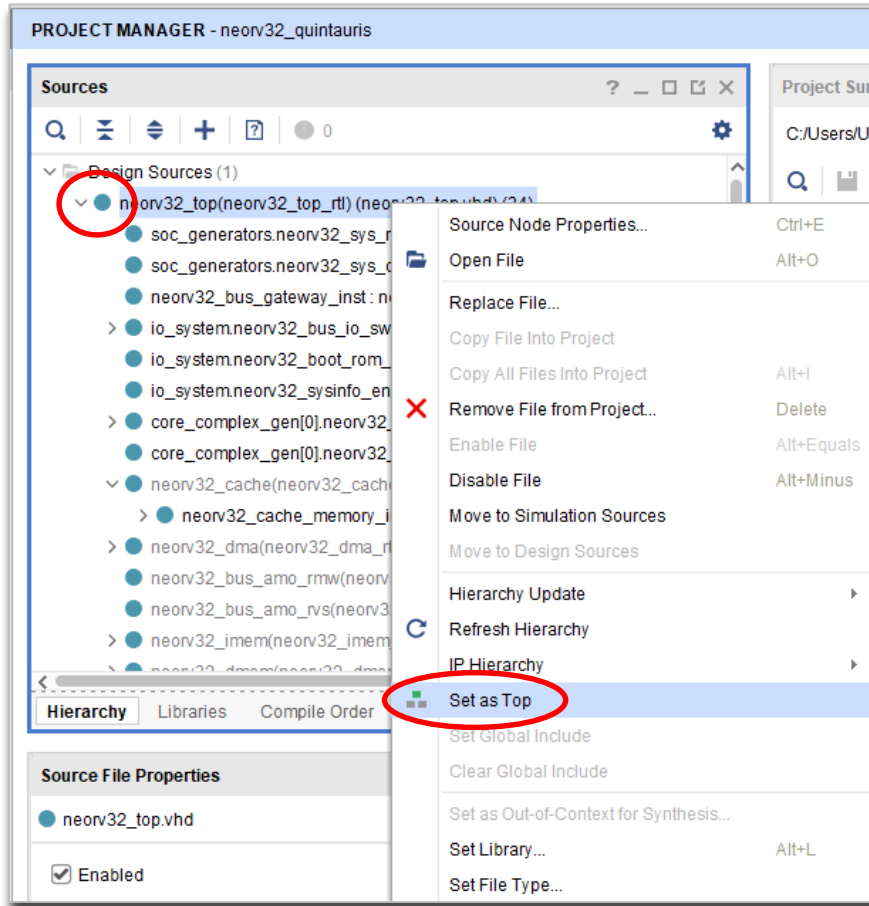


Assign all core VHDL files to neorv32 library



Top-Level module of the project

Set `neorv32_top.vhd` as the top-level



Run the Synthesis of neorv32_top.vhd



▼ SYNTHESIS

▶ Run Synthesis

> Open Synthesized Design

Tcl Console Messages Log Reports Design Runs x ? _ □ ▢																	
Q ⚙ ⚖ ⚡ ⏪ ⏩ ⏴ ⏵ + %																	
Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy
▼ ✓ synth_1	constrs_1	synth_design Complete!								1	1	0.0	0	0	9/5/25, 1:52 PM	00:00:32	Vivado Synthesis Defaults (Vivado Synthesis 20
▶ impl_1	constrs_1	Not started															Vivado Implementation Defaults (Vivado Implem

Neorv_32 Generics (I)



```
entity neorv32_top is
  generic (
    -- Processor Clocking --
    CLOCK_FREQUENCY      : natural          := 0;          -- clock frequency of clk_i in Hz

    -- Dual-Core Configuration --
    DUAL_CORE_EN         : boolean          := false;       -- enable dual-core homogeneous SMP

    -- Boot Configuration --
    BOOT_MODE_SELECT     : natural range 0 to 2      := 0;          -- boot configuration select (default = 0 = bootloader)
    BOOT_ADDR_CUSTOM     : std_ulogic_vector(31 downto 0) := x"00000000"; -- custom CPU boot address (if boot_config = 1)

    -- On-Chip Debugger (OCD) --
    OCD_EN               : boolean            := false;      -- implement on-chip debugger
    OCD_NUM_HW_TRIGGERS  : natural range 0 to 16     := 0;          -- number of hardware break-/watchpoints
    OCD_AUTHENTICATION   : boolean            := false;      -- implement on-chip debugger authentication
    OCD_JEDEC_ID         : std_ulogic_vector(10 downto 0) := "000000000000"; -- JEDEC ID: continuation codes + vendor ID
  );
end entity;
```

Neorv_32 Generics (II)



-- RISC-V CPU Extensions --

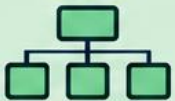
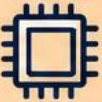
<code>RISCV_ISA_C</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement compressed extension</code>
<code>RISCV_ISA_E</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement embedded RF extension</code>
<code>RISCV_ISA_M</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement mul/div extension</code>
<code>RISCV_ISA_U</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement user mode extension</code>
<code>RISCV_ISA_Zaamo</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement atomic read-modify-write operations extension</code>
<code>RISCV_ISA_Zalrsc</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement atomic reservation-set operations extension</code>
<code>RISCV_ISA_Zcb</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement additional code size reduction instructions</code>
<code>RISCV_ISA_Zba</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement shifted-add bit-manipulation extension</code>
<code>RISCV_ISA_Zbb</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement basic bit-manipulation extension</code>
<code>RISCV_ISA_Zbkb</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement bit-manipulation instructions for cryptography</code>
<code>RISCV_ISA_Zbkc</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement carry-less multiplication instructions</code>
<code>RISCV_ISA_Zbkx</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement cryptography crossbar permutation extension</code>
<code>RISCV_ISA_Zbs</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement single-bit bit-manipulation extension</code>
<code>RISCV_ISA_Zfinx</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement 32-bit floating-point extension</code>
<code>RISCV_ISA_Zicntr</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement base counters</code>
<code>RISCV_ISA_Zicond</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement integer conditional operations</code>
<code>RISCV_ISA_Zihpm</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement hardware performance monitors</code>
<code>RISCV_ISA_Zknd</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement cryptography NIST AES decryption extension</code>
<code>RISCV_ISA_Zkne</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement cryptography NIST AES encryption extension</code>
<code>RISCV_ISA_Zknh</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement cryptography NIST hash extension</code>
<code>RISCV_ISA_Zksed</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement ShangMi block cipher extension</code>
<code>RISCV_ISA_Zksh</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement ShangMi hash extension</code>
<code>RISCV_ISA_Zmmul</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement multiply-only M sub-extension</code>
<code>RISCV_ISA_Zxcfu</code>	<code>: boolean</code>	<code>:= false;</code>	<code>-- implement custom (instr.) functions unit</code>

Neorv_32 setup (I)



Setup of the processor

We need a top-level module that connects the processor to the board's pins.

Concept	What it means	In NEORV32
CPU 	Executes RISC-V instructions.	NEORV32 CPU
SoC 	CPU + memory + peripherals + interconnection.	neorv32_top
FPGA System 	SoC connected to the outside world of the board.	Top-level + XDC + FPGA

Neorv_32 setup (II)



Setup of the processor

- It is necessary to **specify the I/O ports** and **define the Generics** for:
 - Defining the characteristics.
 - Specifying the instruction and data memory.
 - Specifying the processor's instruction set extensions.
- We will use `neorv32_test_setup_bootloader.vhd`
 - located in the `test_setups` folder of the repository.

Neorv_32 setup (III)



Setup of the processor

- `neorv32_test_setup_bootloader.vhd`
 - It is one of the minimal test setups for an initial processor test.
 - It is a demonstration top-level that instantiates and configures `neorv32_top.vhd`.
 - It creates a small NEORV32 SoC that boots using the internal bootloader and communicates with the outside world through GPIO and UART.

Neorv_32 setup (IV)



Setup of the processor

- `neorv32_test_setup_bootloader.vhd`
 - This setup configures a `rv32imc_Zicsr_Zicntr` CPU with 16kB IMEM (as RAM), 8kB DMEM and includes the GPIO module to drive 8 external signals (`gpio_o`), the CLINT module for generating timer interrupts and UART0 to interface with the bootloader or application (via `uart0_txd_o` and `uart0_rxd_i`) via a serial terminal.
 - The setup uses the "indirect boot" configuration, so software applications can be uploaded and run at any timer via the bootloader and a serial terminal.

Neorv_32 setup (V)

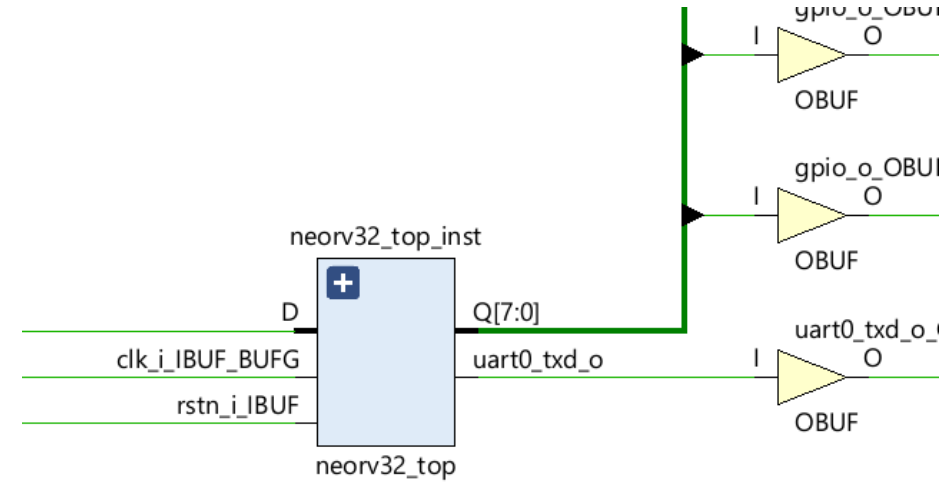


neorv32_test_setup_bootloader.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library neorv32;
use neorv32.neorv32_package.all;

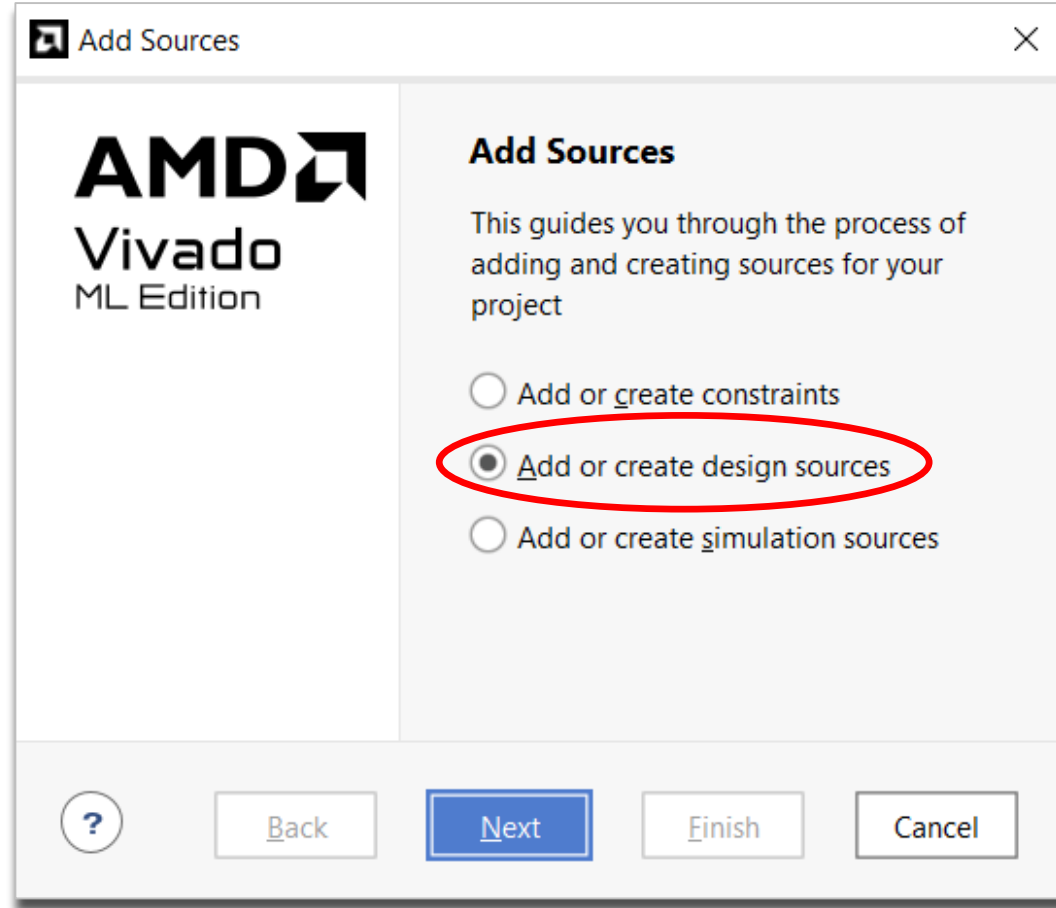
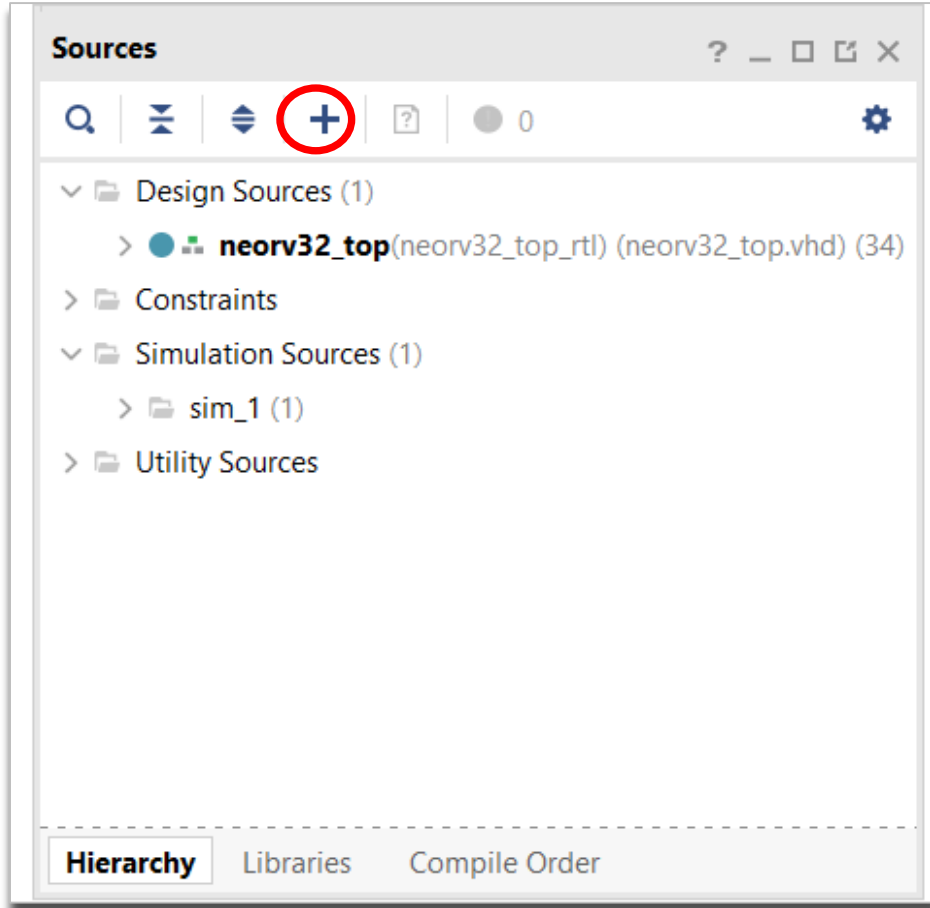
entity neorv32_test_setup_bootloader is
  generic (
    -- adapt these for your setup --
    CLOCK_FREQUENCY : natural := 100000000; -- clock frequency of clk_i in Hz
    IMEM_SIZE        : natural := 16*1024;   -- size of processor-internal instruction memory in bytes
    DMEM_SIZE        : natural := 8*1024     -- size of processor-internal data memory in bytes
  );
  port (
    -- Global control --
    clk_i      : in  std_ulogic; -- global clock, rising edge
    rstn_i     : in  std_ulogic; -- global reset, low-active, async
    -- GPIO --
    gpio_o     : out std_ulogic_vector(7 downto 0); -- parallel output
    -- UART0 --
    uart0_txd_o : out std_ulogic; -- UART0 send data
    uart0_rxd_i : in  std_ulogic  -- UART0 receive data
  );
end entity;
```



Neorv_32 setup (V)



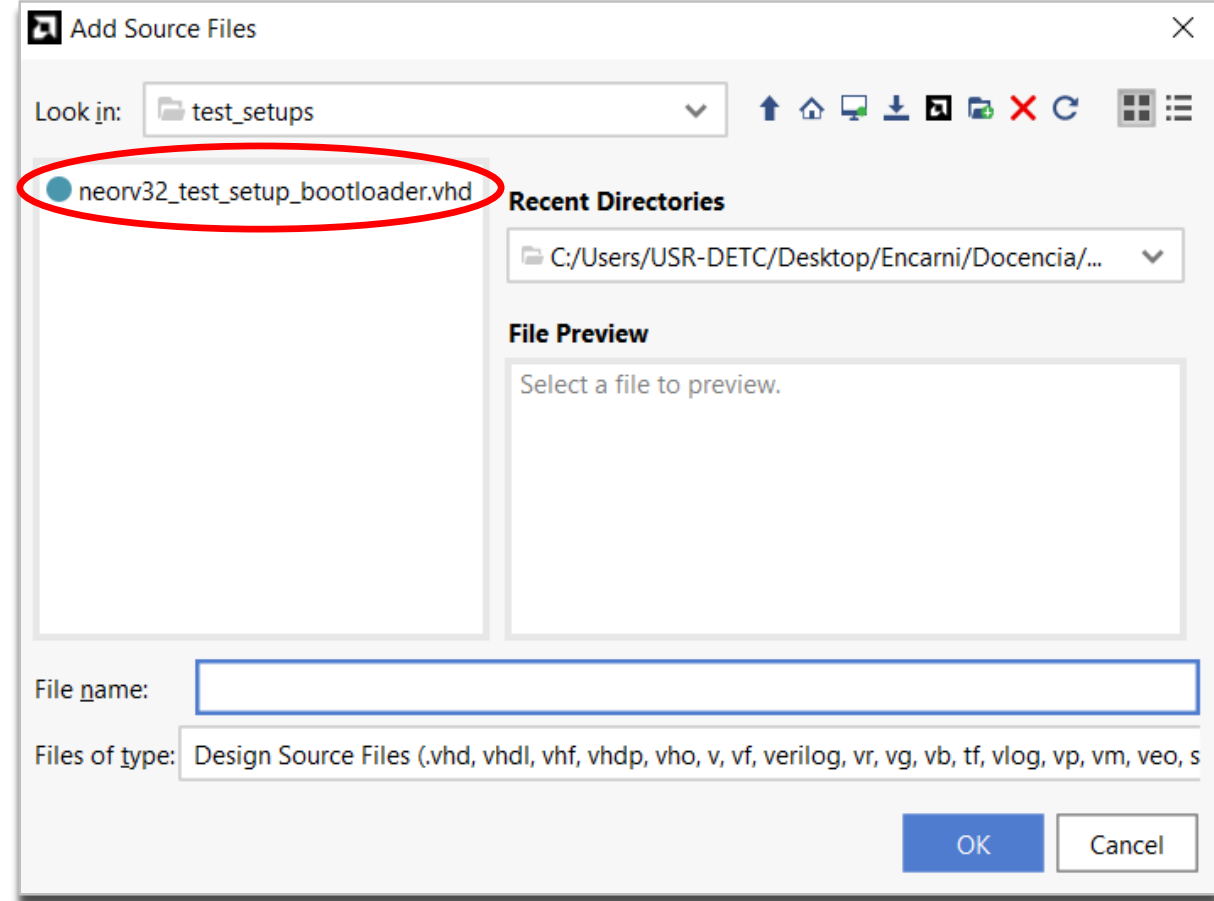
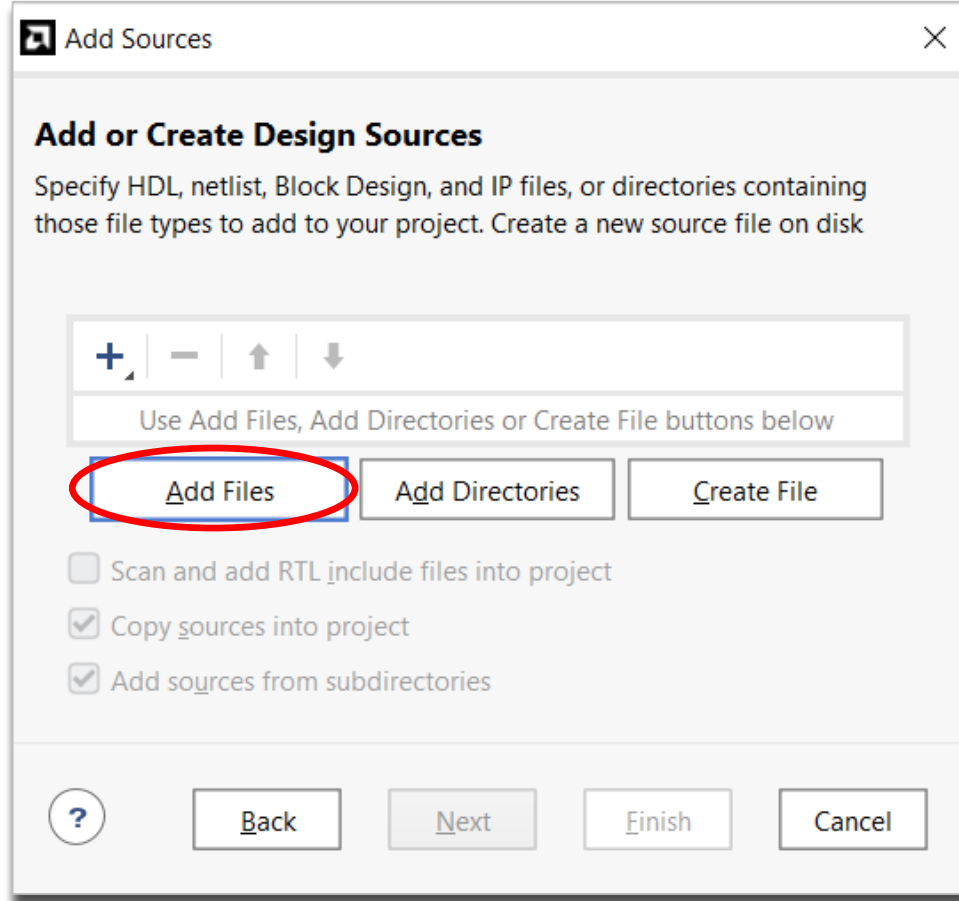
Adding `neorv32_test_setup_bootloader.vhd` **sources to project**



Neorv_32 setup (VI)



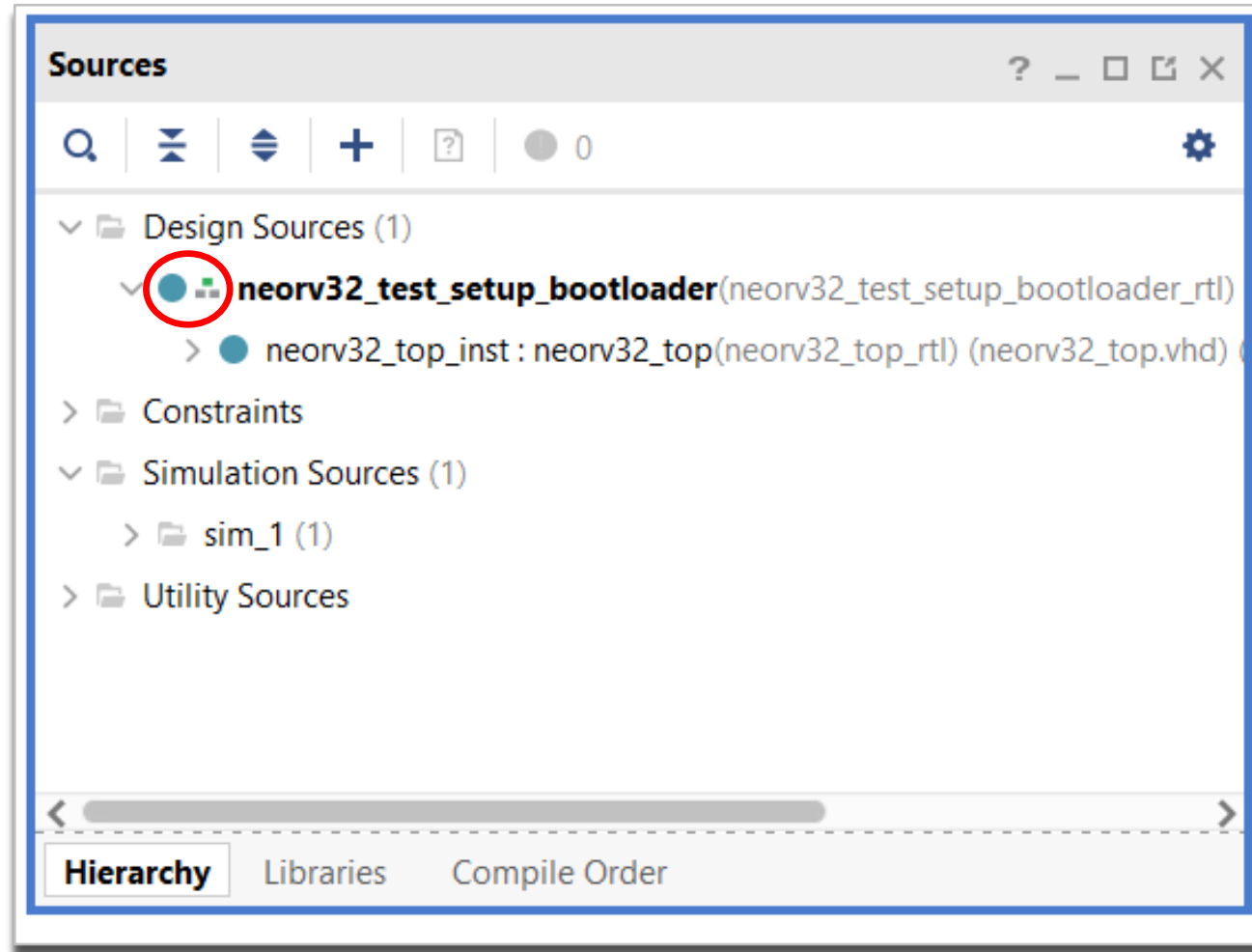
Adding `neorv32_test_setup_bootloader.vhd` **sources to project**



Neorv_32 setup (VII)



Set `neorv32_test_setup_bootloader.vhd` **as the Top-Level Module**



Adapt Neorv_32 to the Arty 7 development board (I)

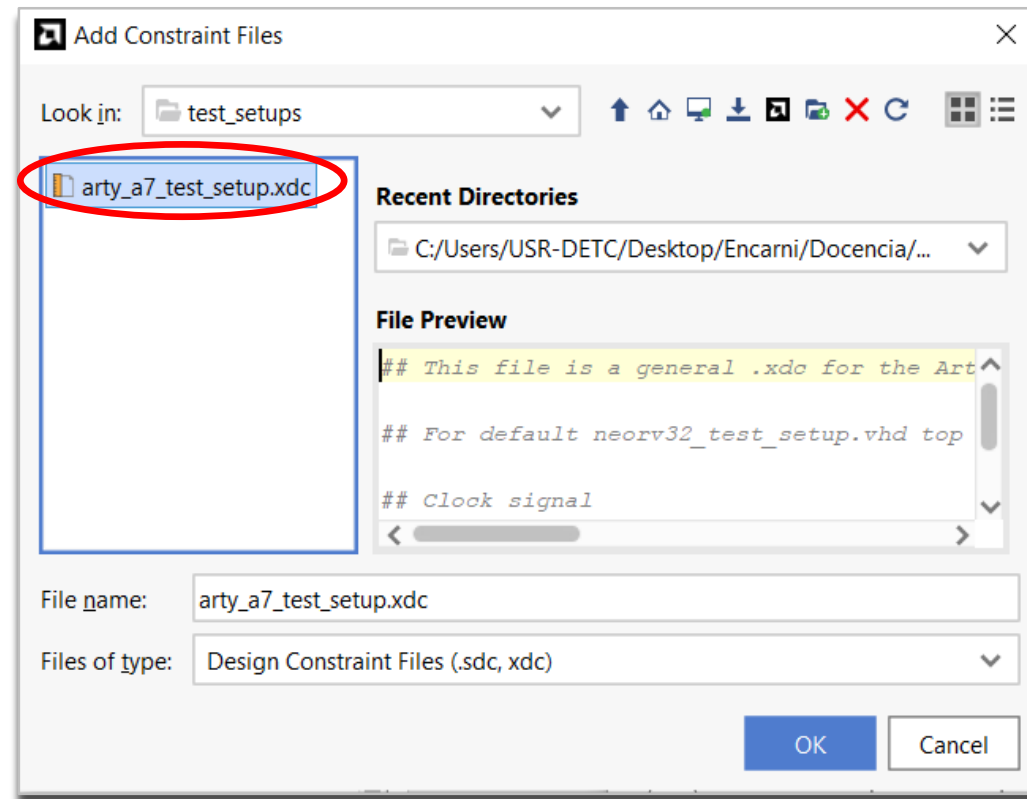
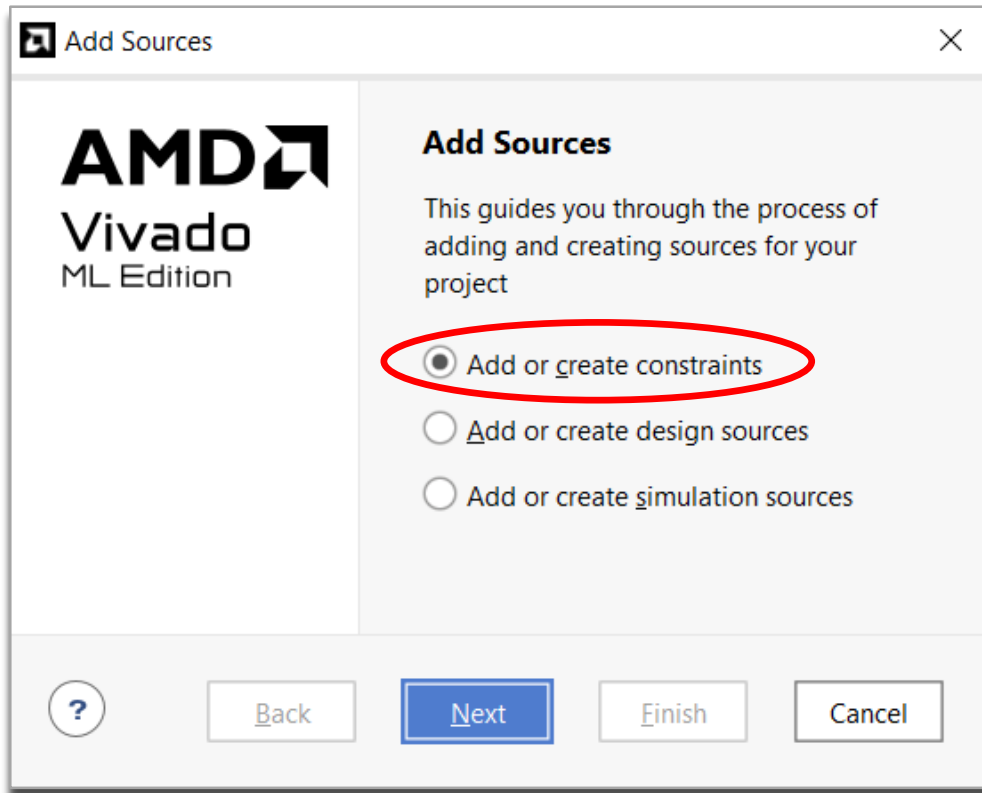


- The design must be adapted to the FPGA device and development board on which the NEORV32 core will be implemented:
 - The NEORV32 repository provides board-specific setups and constraint files: <https://github.com/stnolting/neorv32-setups/tree/main/vivado/arty-a7-test-setup>.
 - From the Vivado folder, we will use the `arty_a7_test_setup.xdc` file. This file must be **added** to the Vivado project as a **Constraints file**.
 - The constraints file defines how each of the RISC-V core's I/O ports is mapped to the corresponding FPGA pins.
 - These FPGA pins are connected to the **UART interface** available through the board's USB port, the **reset button**, and the **four on-board LEDs**.

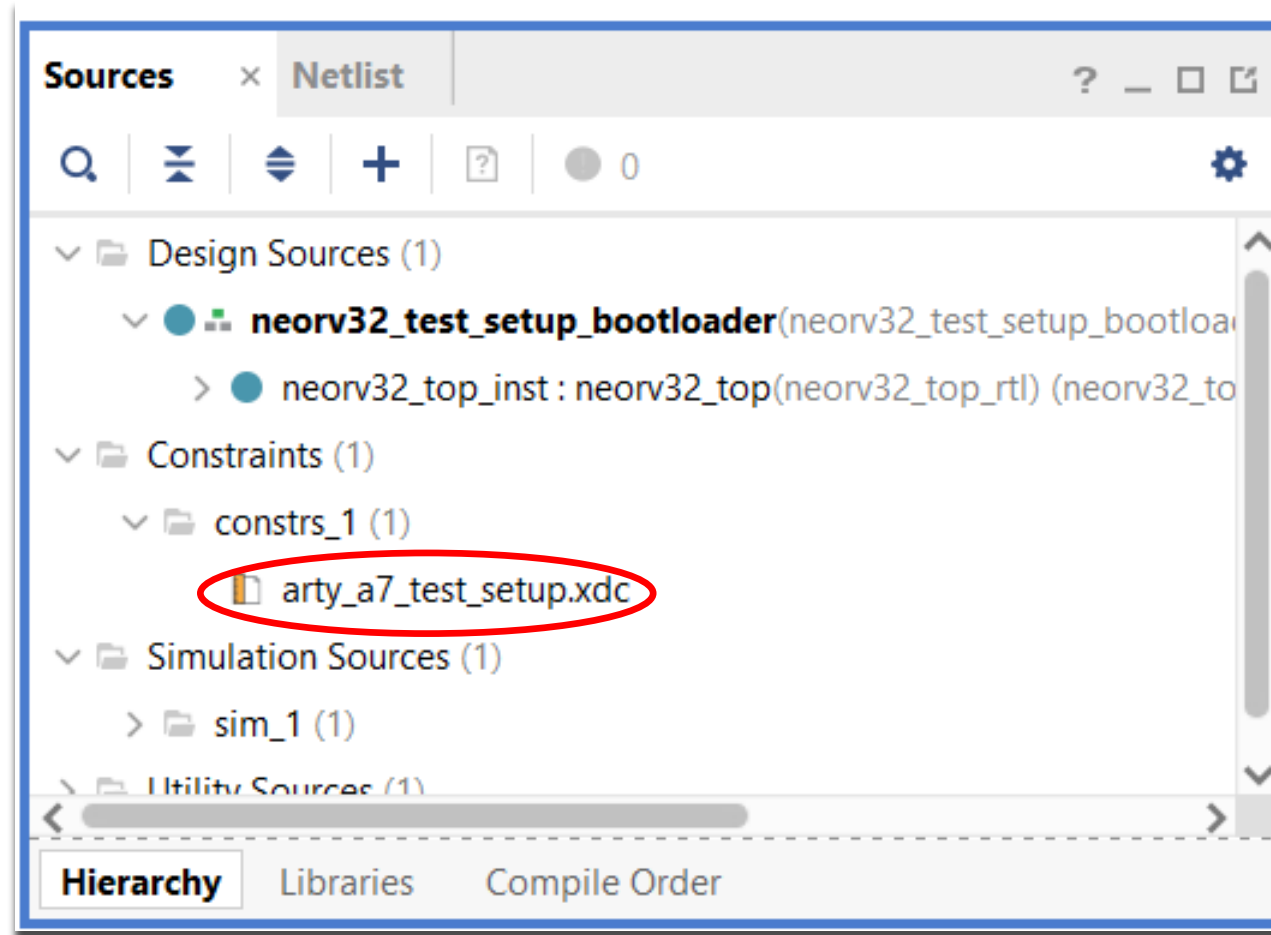
Adapt Neorv_32 to the Arty 7 development board (II)



Add `arty_a7_test_setup.xdc` to the Vivado project as a Constraints file



Adapt Neorv_32 to the Arty 7 development board (III)



Synthesis and Implementation of neorv32_test_setup_bootloader



▼ SYNTHESIS

▶ Run Synthesis

> Open Synthesized Design

▼ IMPLEMENTATION

▶ Run Implementation

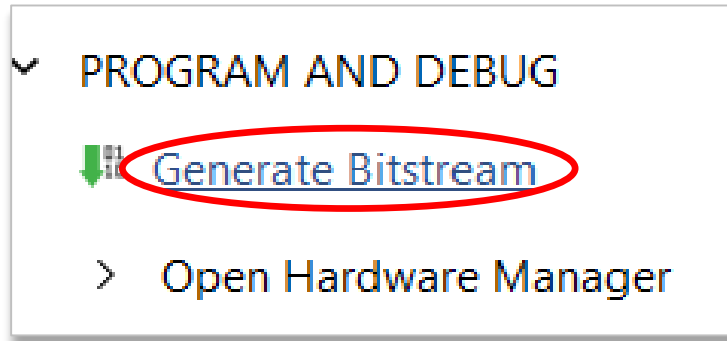
> Open Implemented Design

Tcl Console Messages Log Reports Design Runs × ? _ □ □																	
Q ⌵ ⌶ ⏪ ⏩ ⏴ ⏵ + %																	
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM
▼ ✓ synth_1	constrs_1	synth_design Complete!												2027	1455	8	0
✓ impl_1	constrs_1	route_design Complete!	0.950	0.000	0.059	0.000		0.000	0.119	0	21 Warn			2005	1465	8	0

Bitstream Generation and FPGA Programming (I)



- Once synthesis and implementation have been successfully completed, generate the bitstream for the FPGA.

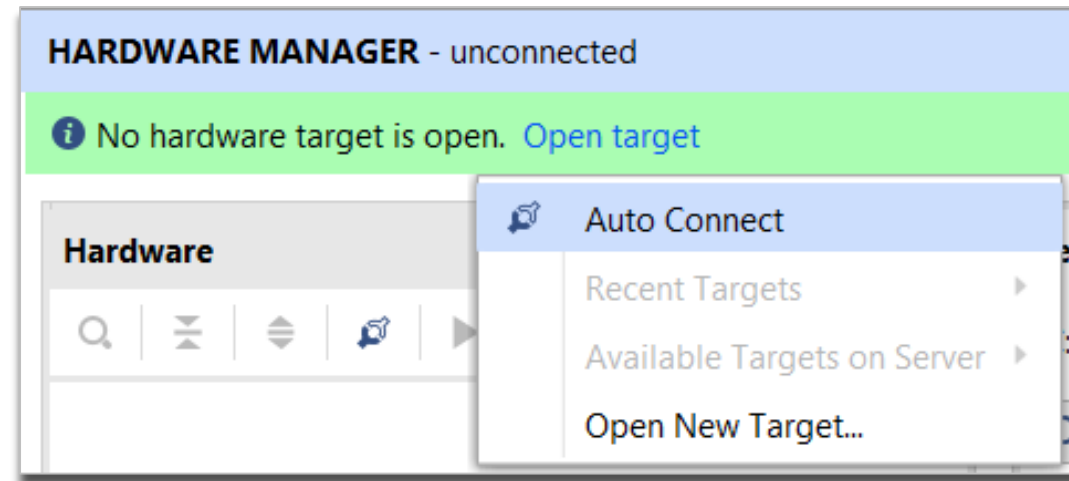
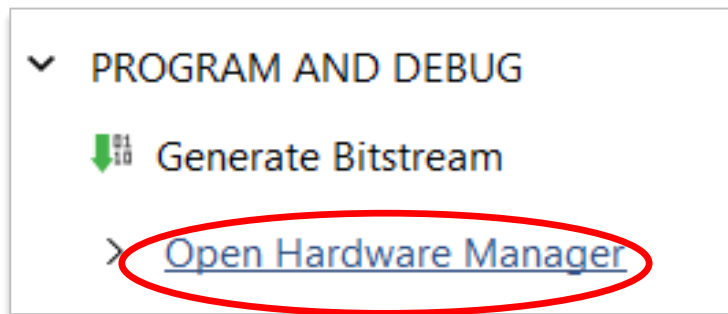


Tcl Console	Messages	Log	Reports	Design Runs	DRC	Methodology	Power	Timing
Q ⌵ ⌶ ⌷ ⏪ ⏩ ⏴ ⏵ ⏶ ⏷								
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS
✓ synth_1	constrs_1	synth design Complete!						
✓ impl_1	constrs_1	write_bitstream Complete!	0.845	0.000	0.077	0.000		0.000

Bitstream Generation and FPGA Programming (II)



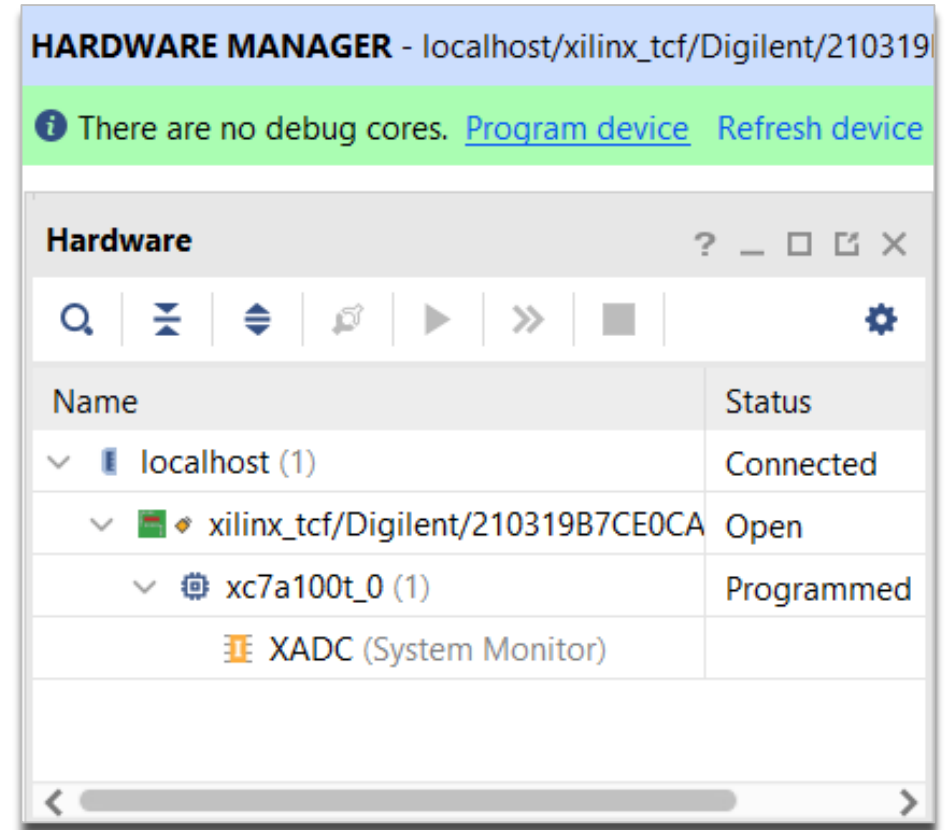
- Open the **Hardware Manager** in Vivado.
- Connect the Arty A7-100T development board to the PC using the USB cable
- In the Hardware Manager, select `Open Target` → `Auto Connect`.
- Vivado should automatically detect the FPGA device connected to the PC.
- The detected device should then appear in the Hardware Manager, ready to be programmed with the generated bitstream.



Bitstream Generation and FPGA Programming (III)



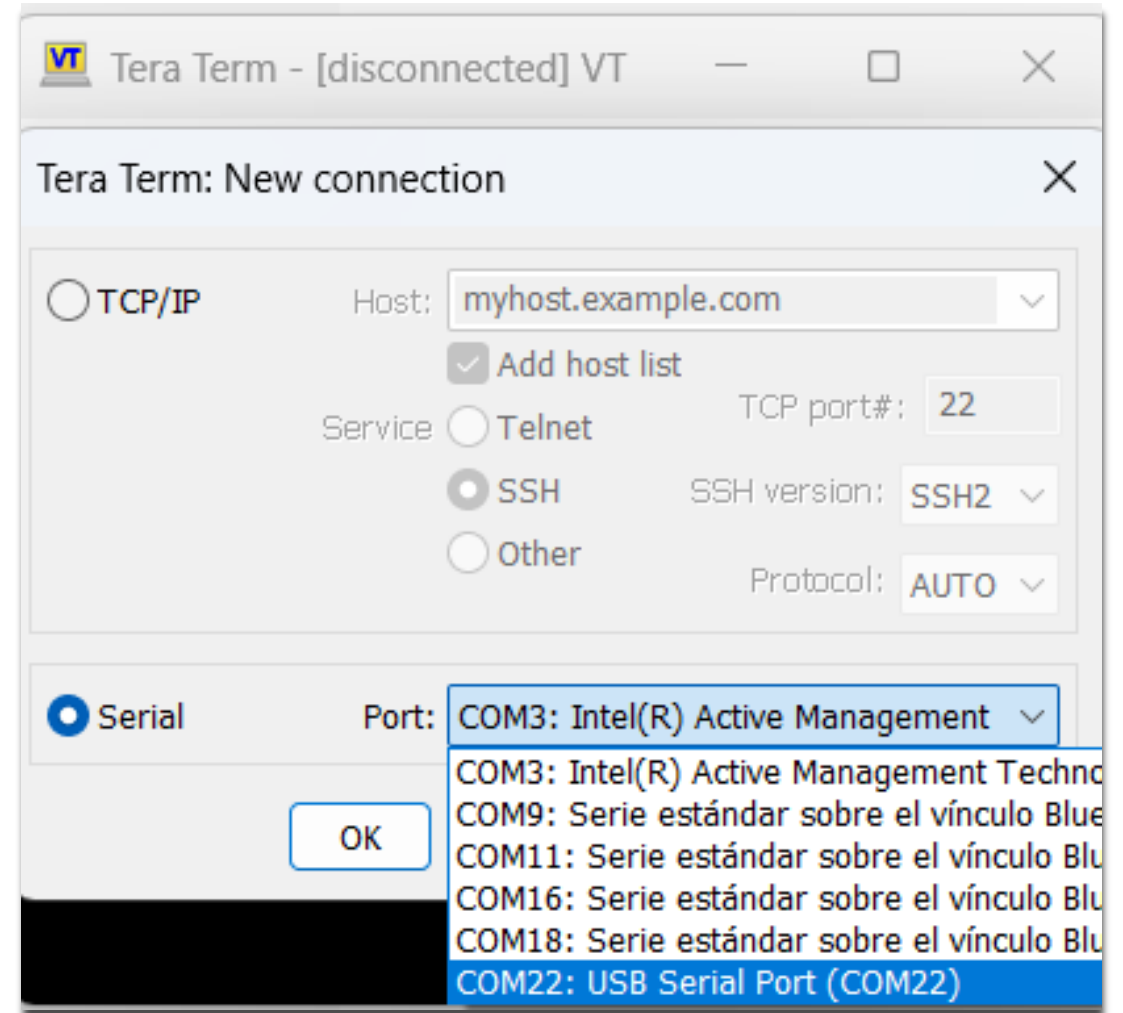
- Click **Program Device** to open the window for selecting the **bitstream file** to program the FPGA.
- Select the bitstream file shown by default and click **OK**.
- Once programming is complete:
 - the **NEORV32 RISC-V processor** will be running on the FPGA.
 - **LD0 LED should start blinking.**



UART Connection to the NEORV32 Core (I)



- Download and install **Tera Term**.
- Check which **COM port** the board is connected to.
- Set the baud rate to **19,200 bps**.



UART Connection to the NEORV32 Core (II)



- Configure Tera Term display (size and font size)

The screenshot shows the 'Tera Term: Additional settings' dialog box with the 'Terminal' tab selected. The 'Terminal size' section has a width of 120 and a height of 24. The 'Term size = win size' checkbox is checked, and 'Auto window resize' is unchecked. The 'New-line' section has 'Receive' and 'Transmit' both set to 'CR'. The 'Terminal ID' is set to 'VT100', and 'Local echo' is unchecked. The 'Answerback' field is empty, and 'Auto switch (VT<->TEK)' is unchecked.

General	Control Sequence	Copy and Paste	Visual	Log	Cygwin	Encoding	
Font	Theme	Keyboard	Mouse	TCP/IP	Terminal	Serial port	Window

Terminal size

120 x 24

☒ Term size = win size

☐ Auto window resize

New-line

Receive: CR

Transmit: CR

Terminal ID: VT100

☐ Local echo

Answerback:

☐ Auto switch (VT<->TEK)

UART Connection to the NEORV32 Core (III)



- The **bootloader prompt** should now be displayed.
- The available commands can be listed by **pressing** "h".
- Applications can be uploaded to the processor using the bootloader.
- To do so, the **applications** must first be **compiled** using the appropriate toolchain.

```
CMD:> r

NEORV32 Bootloader
build: Sep  3 2025

Auto-boot in 10s. Press any key to abort.
Aborted.

Type 'h' for help.
CMD:> h
Available CMDs:
h: Help
i: System info
r: Restart
u: Upload via UART
l: SPI flash - load
s: SPI flash - program
e: Start executable
x: Exit
CMD:> █
```

Toolchain Compilation (Linux)

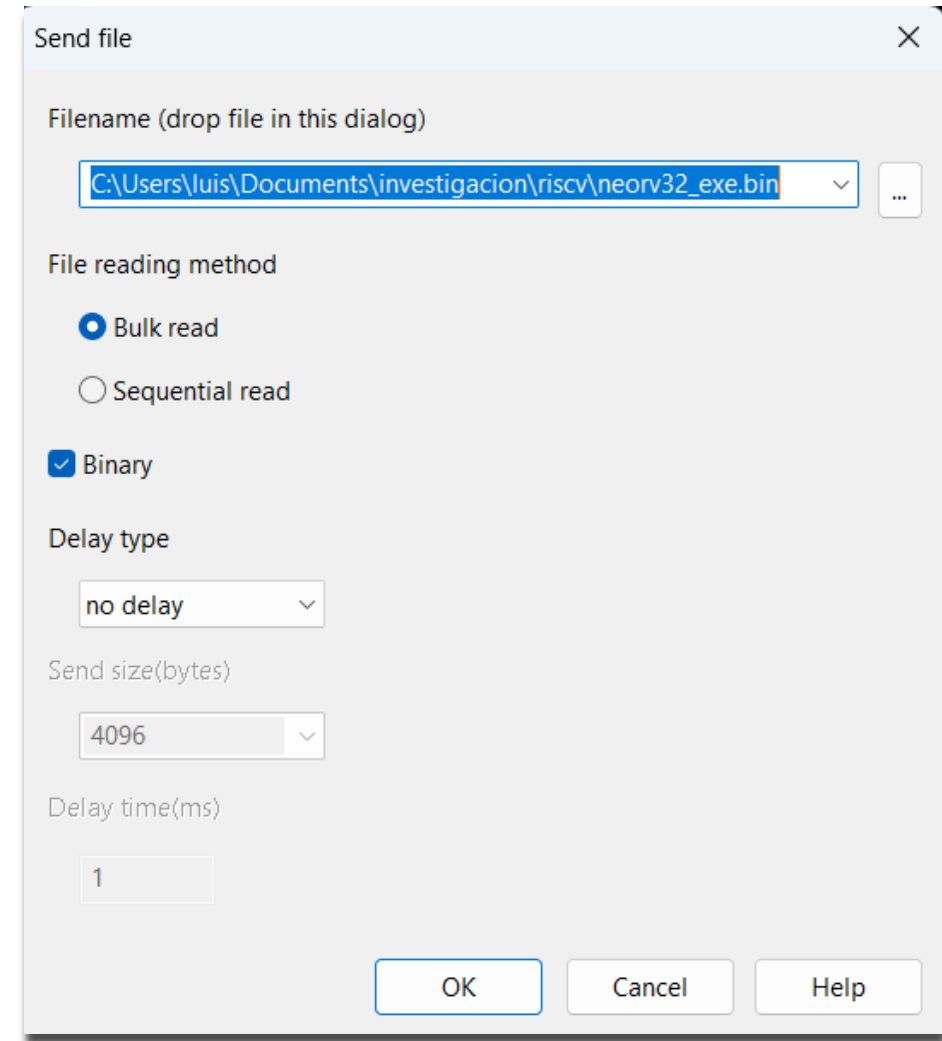


- `sudo apt update`
- `sudo apt install -y autoconf automake autotools-dev
curl python3 \
libmpc-dev libmpfr-dev libgmp-dev gawk build-essential
bison flex \
texinfo gperf libtool patchutils bc zlib1g-dev git
libexpat1-dev`
- `git clone https://github.com/riscv/riscv-gnu-toolchain`
- `cd riscv-gnu-toolchain`
- `sudo ./configure --prefix=/opt/riscv --with-
arch=rv32imac_zifencei_zicsr --with-abi=ilp32`
- `make`

Loading and Executing a Binary



- Compile the corresponding **C program** (covered in Part II).
- Use the bootloader's "**u**" **command**.
- **Upload the executable program** using **Tera Term**. In this case, use the provided "**Hello World**" program.
- Once the upload is complete, use the bootloader's "**e**" **command**.



Execution Results



```
NEORV32 Bootloader  
build: Sep  3 2025
```

```
Auto-boot in 10s. Press any key to abort.  
Aborted.
```

```
Type 'h' for help.
```

```
CMD:> u
```

```
Awaiting neorv32_exe.bin... OK
```

```
CMD:> e
```

```
Booting from 0x00000000...
```

```
##          ##  #####          #####          #####  ##  ##  #####          #####  ##          ##  ##  ##  
####      ##  ##          ##          ##  ##          ##  ##  ##          ##  ##          ##  ##          ##  
## ##      ##  ##          ##          ##  ##          ##  ##          ##          ##          ##          ##  
##  ##      ##  #####          ##          ##  #####          ##          ##          ##          ##          ##  
##          ##  ##          ##          ##  ##          ##          ##          ##          ##          ##          ##  
##          ####  ##          ##          ##          ##          ##          ##          ##          ##          ##  
##          ##  #####          #####          ##          ##          ##          #####          #####  
##          ##          #####          #####          ##          ##          ##          #####          #####  
##          ##          ##          ##          ##          ##          ##          ##          ##          ##          ##
```

```
Hello world! :)
```

Compilation of the Hello_world example



- Go to the corresponding folder (~/`neorv32-main/neorv32-main/sw/example/hello_world`)
- Modify makefile, adding/entering:

```
# Override the default CPU ISA  
MARCH = rv32i_zicsr_zifencei
```

```
# Override the default RISC-V GCC prefix  
RISCV_PREFIX ?= riscv32-unknown-elf-
```

- Define the following environment variables:

```
export RISCV=/opt/riscv  
export PATH=$RISCV/bin:$PATH  
export CROSS_COMPILE=riscv32-unknown-elf-  
export CC=${CROSS_COMPILE}gcc  
export OBJCOPY=${CROSS_COMPILE}objcopy
```
- Compile with “make all”
- From all the generated executables, use `neorv32_exe.bin`
- Load and execute this file using Teraterm and the bootloader.

Execution Results



```
NEORV32 Bootloader
build: Sep  3 2025

Auto-boot in 10s. Press any key to abort.
Aborted.

Type 'h' for help.
CMD:> u
Awaiting neorv32_exe.bin... OK
CMD:> e
Booting from 0x00000000...
```

```
##          ##  #####          #####          #####  ##  ##  #####          #####  ##          ##  ##  ##
####         ##  ##          ##          ##  ##          ##  ##          ##  ##          ##  ##          ##  ##
## ##        ##  ##          ##          ##  ##          ##  ##          ##  ##          ##  ##          ##  ##
##  ##       ##  ##          ##          ##  ##          ##  ##          ##  ##          ##  ##          ##  ##
##   ##  ##  ##          ##          ##  ##          ##  ##          ##  ##          ##  ##          ##  ##
##   ####  ##          ##          ##  ##          ##  ##          ##  ##          ##  ##          ##  ##
##   ##          #####          ##          ##          ##          ##          ##          ##          ##          ##
##          ##          ##          ##          ##          ##          ##          ##          ##          ##          ##
Hello world! :)
```


Compilation of Hello_world example



- Go to the corresponding folder (~/neorv32-main/neorv32-main/sw/example/hello_world)
- Modify makefile, adding/entering:

```
# Override the default CPU ISA  
MARCH = rv32i_zicsr_zifencei
```

```
# Override the default RISC-V GCC prefix  
RISCV_PREFIX ?= riscv32-unknown-elf-
```

- Define the following environment variables:

```
export RISCV=/opt/riscv  
export PATH=$RISCV/bin:$PATH  
export CROSS_COMPILE=riscv32-unknown-elf-  
export CC=${CROSS_COMPILE}gcc  
export OBJCOPY=${CROSS_COMPILE}objcopy
```
- Compile with "make all"
- From all the generated executables, use neorv32_exe.bin
- Load and execute this file using Teraterm and the bootloader.

Exercise: Compile and execute performance_tests / I / M / Zfinx



- Problem: Exception when uploading program
- Not enough memory to load the program.

```
COM22 - Tera Term VT
Archivo  Editar  Configuracion  Control  Ventana  Ayuda
NEORV32 Bootloader
build: Jan 22 2026

Auto-boot in 8s. Press any key to abort.
Aborted.

Type 'h' for help.
CMD:> u
Awaiting neorv32_exe.bin...
ERROR_EXCEPTION 0x00000007 0xffe002fc 0x00c72023 0x00004000
```

Exercise: Compile and execute performance_tests / I / M / Zfinx



- Modify neorv32 processor to extend instruction and data memories to 128KB
- Synthesize, implement and generate the new bitstream
- Program the FPGA with the new neorv32 design.
- Upload and execute performance tests for I instructions

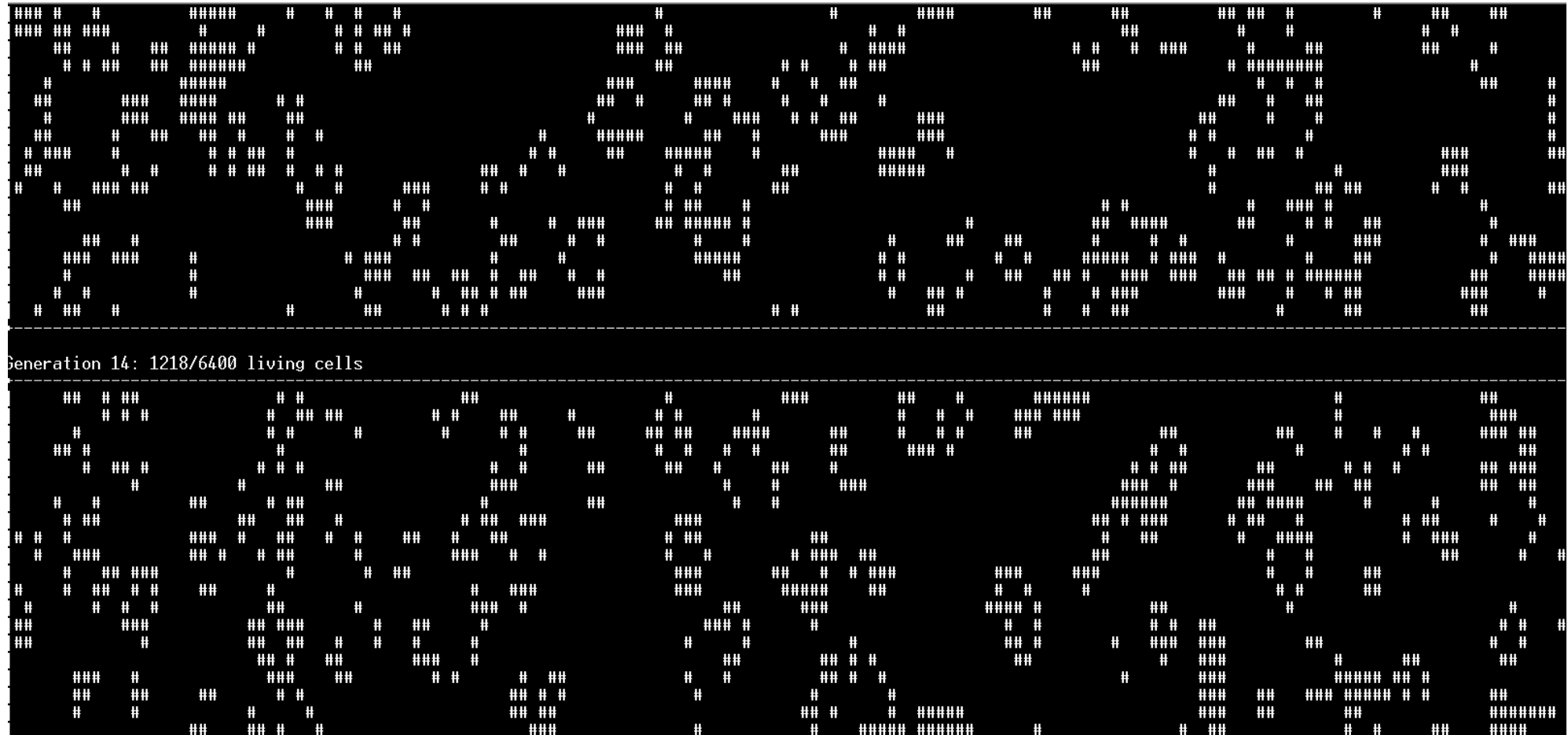
```
entity neorv32_test_setup_bootloader is
  generic (
    -- adapt these for your setup --
    CLOCK_FREQUENCY : natural := 100000000;
    IMEM_SIZE       : natural := 128*1024;
    DMEM_SIZE       : natural := 128*1024
  )
end entity
```

```
Type 'h' for help.
CMD:> u
Awaiting neorv32_exe.bin... OK
CMD:> e
Booting from 0x00000000...

<<< I performance test >>>
SILENT_MODE enabled (only showing per instruction totals)

perform: for (i=0;i<1,i++) {256 instructions}
add rd,rs1,rs2 inst. 2 cyc
addi rd,rs1,imm inst. 2 cyc
sub rd,rs1,rs2 inst. 2 cyc
lui rd,imm inst. 2 cyc
auipc rd,imm inst. 2 cyc
```

Exercise: Compile and execute game of life



·QUINTAURIS

quintauris.com